



Research Article

SimBlockLink: A middleware for linking IoT simulations with real blockchain platforms for enhanced performance evaluation

Ali Alzubaidi ^{a, ID}, Adel Albshri ^{b, ID}, Karan Mitra ^{c, ID}, Rajiv Ranjan ^{b, ID}, Ellis Solaiman ^{b, ID, *}^a Computers Department, College of Engineering and Computing in Al-Qunfudhah, Umm Al-Qura University, Makkah 21955, Saudi Arabia^b School of Computing, Newcastle University, Newcastle upon Tyne NE4 5TG, United Kingdom^c Department of Computer Science, Electrical and Space Engineering, Luleå University of Technology, Skellefteå 931 87, Sweden

ARTICLE INFO

Keywords:

Blockchain
IoT
Middleware
Simulation
Hyperledger
Performance
Benchmarking

ABSTRACT

Recently, there has been growing interest in the research community in exploring the potential of blockchain technology for various Internet of Things (IoT)-based domains, such as smart cities. While several open-source blockchain platforms like Hyperledger Fabric are readily available, integrating them into large-scale IoT infrastructures remains challenging due to factors such as performance, cost, and scalability. Simulation studies provide a viable option for carefully assessing the applicability of blockchains in dynamic, large-scale, and heterogeneous IoT scenarios. In theory, integrating IoT simulators with actual blockchain platforms is possible. However, the challenge lies in addressing the disparity between the distinctive environments of IoT simulators and blockchain platforms. This paper introduces a novel middleware architecture, SimBlockLink, that overcomes these challenges in enabling interaction between blockchain platforms and IoT simulators. SimBlockLink bridges the distinct operational facets of both, supporting thorough evaluations of real blockchain-based solutions in simulated IoT scenarios. We present a study where SimBlockLink connects IoTsim-Osmosis, an IoT simulation tool, with an actual Hyperledger Fabric blockchain platform. To demonstrate scalability, we conduct a large-scale city firefighting case study involving 30,000 IoT devices, validating SimBlockLink's ability to sustain high-volume transaction loads. In addition, we provide a security and privacy analysis of the middleware, outlining threat models and built-in mechanisms such as transport layer security (TLS) channels, concurrency control, and fail-safe logic to ensure robust and dependable operation. Our results demonstrate that SimBlockLink enables effective interaction between blockchain deployments and simulated IoT models, ensuring smooth interoperability and efficient evaluation of key blockchain performance metrics, including throughput, latency, and transaction success/failure rates.

1. Introduction

The Internet of Things (IoT) has emerged as a transformative technology, connecting billions of devices and enabling innovative applications in various domains, such as healthcare, transportation, manufacturing, and smart cities [1]. IoT devices, equipped with sensors and communication capabilities, generate large amounts of data that can be analyzed to derive valuable insights and optimize processes. Blockchain has been seen as a promising technology for addressing IoT issues and even transforming how IoT solutions are designed and deployed [2]. Blockchain offers several advantages: decentralization, high availability, mitigation of single points of failure, immutability, transparency, and traceabil-

ity [3]. These features enable secure data sharing among untrusted parties, prevent tampering with stored data, and provide a verifiable audit trail for IoT transactions [4]. Recently, there has been an increasing interest in the convergence between IoT and blockchain to unlock the potential of both paradigms [5].

Research studies on integration between blockchain and IoT deal with several issues, such as limited computational power, energy, or bandwidth for IoT devices to participate in blockchain networks [6,7]. Furthermore, the decentralized nature of blockchain can result in performance overhead and scalability issues, especially when handling high transaction volumes and real-time data processing requirements [8]. In addition, designing and investigating a proper blockchain-based IoT so-

* Corresponding author.

E-mail address: ellis.solaiman@newcastle.ac.uk (E. Solaiman).<https://doi.org/10.1016/j.bcr.2025.100374>

Received 16 June 2024; Received in revised form 1 May 2025; Accepted 19 May 2025

lution is a nontrivial task. This is mainly due to the need for sufficient access to Blockchain and IoT technologies. One can find that blockchain platforms are open-source, which allows researchers to experiment with diverse blockchain scenarios. However, implementing IoT scenarios requires access to large-scale infrastructure, which can be both costly and resource-intensive. These constraints limit researchers' ability to conduct comprehensive performance evaluations under varying conditions, emphasizing the importance of benchmark tools to mitigate these challenges.

In this context, simulation emerges as a viable benchmarking approach, providing a controlled environment for evaluating large-scale IoT systems without the need for physical infrastructure. Several IoT simulators, including SimIoT [9], Edge-Fog [10], EdgeCloudSim [11], and IoTsim-Osmosis [12], provide frameworks for modelling IoT systems, encompassing physical devices, edge computing, networks, data centres, and cloud services [13]. However, a limitation of existing simulators is their inability to interface with real blockchain platforms, as they function in isolated environments. This gap prevents accurate evaluation of blockchain-IoT interactions, creating a substantial obstacle in developing realistic benchmark frameworks.

To bridge this gap, a middleware solution is required to facilitate seamless integration between IoT simulators and blockchain platforms. Middleware can act as an interface layer, enabling synchronized data exchange and realistic transaction processing, thereby enhancing the accuracy of Blockchain-based IoT performance evaluations. However, designing such middleware introduces several key challenges in aligning IoT simulations with real blockchain platforms. These challenges include:

- **Execution environment differences:** Most IoT simulators rely on discrete event modeling, where events are processed sequentially within a short-lived virtual timeframe. In contrast, blockchain platforms operate as distributed systems with continuous transaction processing and dynamic state changes. A naive integration of these disparate environments can result in misalignment between simulated IoT behaviors and real blockchain operations.
- **Real-time data synchronization:** IoT simulators generate data asynchronously, whereas blockchain platforms process transactions in real-time. This mismatch can lead to data misalignment, affecting the accuracy of performance evaluation.
- **Concurrency and race conditions:** When multiple simulated IoT devices generate transactions simultaneously, race conditions and data conflicts may arise.
- **Error handling and fault tolerance:** Failures in communication, transaction processing, or smart contract execution, can disrupt simulations, requiring robust error-handling mechanisms.

To address these challenges, this paper introduces *SimBlockLink*, a novel middleware that seamlessly bridges IoT simulators and blockchain platforms. SimBlockLink enables real-time interaction, allowing for the evaluation of critical blockchain performance metrics, including throughput, latency, and transaction success/failure rates. This paper makes the following **key contributions**:

1. **Modular middleware architecture:** SimBlockLink is designed to be compatible with various simulation environments, particularly Java-based IoT simulators and blockchain platforms. Its architecture follows a separation-of-concerns approach, where agents handle IoT events while workers process blockchain transactions independently. A concurrent storage mechanism buffers IoT-generated metrics for real-time synchronization. Additionally, a lock-based concurrency control ensures data integrity by preventing race conditions, while a fail-safe mechanism retries failed transactions and logs persistent failures without disrupting execution.

2. **Benchmarking:** Real-time performance metrics, such as throughput, latency, and transaction success/failure rates, are monitored using Grafana/ Prometheus.
3. **Validation:** The effectiveness of SimBlockLink is demonstrated through a firefighting case study, where 30,000 smart homes are simulated using IoTsim-Osmosis [12]. Fire alerts are transmitted via SimBlockLink and processed on Hyperledger Fabric [14], showcasing seamless IoT-blockchain integration at scale.
4. **Verification:** SimBlockLink undergoes testing through three structured approaches:
 - (a) **Standalone testing**– Evaluates the middleware independently, without an IoT simulator or blockchain platform.
 - (b) **Blockchain-side testing**– Interfaces with the blockchain to verify correct operation.
 - (c) **Full integration testing**– Connects to both an IoT simulator and a blockchain platform to assess overall functionality and correctness.
5. **Security and privacy analysis:** We provide an analysis of threat models relevant to IoT-blockchain integration, and outline the built-in mechanisms in SimBlockLink (e.g., TLS-secured channels, lock-based concurrency control, and fail-safe transaction handling) that ensure robustness and dependability.
6. **Open-source reference implementation:** This paper contributes a reference implementation of the proposed middleware (SimBlockLink), which is publicly available on GitHub.¹

The remainder of this paper is structured as follows: Section 2 outlines the convergence of IoT and blockchain technologies, discussing their challenges and opportunities. Section 3 presents the proposed SimBlockLink architecture, detailing its key components and design principles. Section 4 describes the implementation of SimBlockLink, including its middleware components and integration mechanisms. Section 5 evaluates SimBlockLink by validating the correctness of the concurrent storage mechanism and demonstrating its integration with Hyperledger Fabric and IoTsim-Osmosis through a case study. Section 6 reviews related works, focusing on IoT-blockchain convergence and current experimentation methods. Finally, Section 7 concludes the paper and outlines future research directions.

2. Background

This section provides key concepts of blockchain (Section 2.1) and IoT (Section 2.2). Subsequently, it highlights the motivations for integrating these technologies (Section 2.3) and discusses the associated challenges (Section 2.4). Finally, it presents performance evaluation approaches (Section 2.5).

2.1. Blockchain technology

In centralized systems, a single authority usually processes transactions between parties. With this model, various challenges must be dealt with, including security vulnerabilities (e.g., unauthorized access), reliability concerns (e.g., single point of failure), trust levels, and misconduct acts. Blockchain technology was introduced with the promise of overcoming these challenges by allowing untrusted parties to interact in a distributed manner without the need for a third party. Unlike centralized systems, a blockchain system is essentially a decentralized system that processes transactions by an interconnected network of independent computing nodes. Every processed transaction is immutably recorded within blocks on a replicated ledger over participating nodes.

The participating nodes are responsible for maintaining the validity of the commonly shared ledger by collecting transactions from a pool of

¹ SimBlockLink Repository: <https://github.com/aakzubaidi/BMBmid-Middleware>.

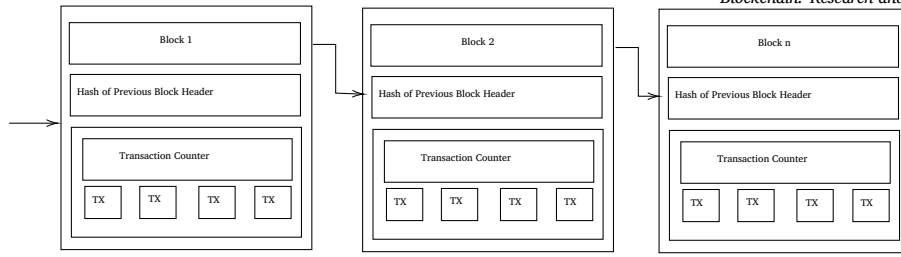


Fig. 1. A chain of blocks.

pending transactions, validating the picked transactions, and appending them to new blocks. Each validating node proposes its block of transactions by propagating it to other peers for consensus on the validity of the block's structure and content. Ultimately, a successfully validated block will end up appended to the ledger of each participating node.

Blockchain derives its name from its ledger structure, which organizes its transactions into a chain of blocks, as depicted in Fig. 1. Each block is uniquely identified by a cryptographic hash and linked to its predecessor, forming a series of interlinked blocks. Once a block is generated and appended to the blockchain ledger, misbehaving nodes find it difficult to alter transactions without rewriting all preceding blocks. This property makes the blockchain system resistant to tampering and the double-spending problem [15]. The unique blockchain data structure promotes transparency and traceability while ensuring high availability through data replication across participating nodes.

Bitcoin [16] is the first and most well-known blockchain example, which solely serves the purpose of being a decentralized cash system over a peer-to-peer network. Bitcoin lacks the openness and programming capabilities needed to develop further decentralized applications beyond financial use cases. Consequently, several blockchains emerged to remedy this shortcoming by offering agnostic blockchain infrastructure and smart contracts. Hyperledger Fabric [14], Ethereum [17], and Quorum [18] are common examples of blockchain platforms for decentralized applications (DApp). The concept of smart contracts plays an essential role in these blockchain platforms, which is a computer program that can be incorporated into the blockchain to provide unified instructions to the participating nodes about processing and recording transactions and their payload data. Consequently, it has been possible to seamlessly integrate blockchain with various technologies such as the IoT and Cloud computing to serve a multitude of DApps in several domains such as education, healthcare, supply chain, manufacturing, and others.

While most blockchain platforms emphasize the concept of decentralization, they can be categorized into three main types of network openness: public, consortium, and private [19]. Each type has its own unique characteristics, advantages, and disadvantages, as illustrated in Table 1.

1. **Public blockchains:** (e.g., Bitcoin [16], Ethereum [17]) allow unrestricted participation, offering transparency and decentralization but facing scalability, privacy, and energy consumption challenges.
2. **Consortium blockchains:** (e.g., Hyperledger Fabric [14]) restricts participation to authenticated entities, improving efficiency, privacy, and governance while maintaining partial decentralization.
3. **Private blockchains:** is fully controlled by a single entity, optimizing efficiency and confidentiality, but limiting decentralization.

The selection among public, consortium, and private blockchains depends on their suitability for various use cases and requirements. Public blockchains are well-suited for applications that emphasize decentralization, transparency, and security. Private blockchains are most suitable for applications that prioritize efficiency, confidentiality, and governance. Consortium blockchains are optimal for applications that

Table 1

Comparison of the key features of public, consortium, and private blockchains.

Feature	Public blockchain	Consortium blockchain	Private blockchain
Access	Permissionless	Permissioned	Permissioned
Decentralisation	High	Medium	Low
Transparency	High	Medium	Low
Performance	Low	Medium	High
Privacy	Low	Medium	High
Governance	Decentralised	Partially centralised	Centralised
Examples	Bitcoin, Ethereum	Hyperledger Fabric	Quorum

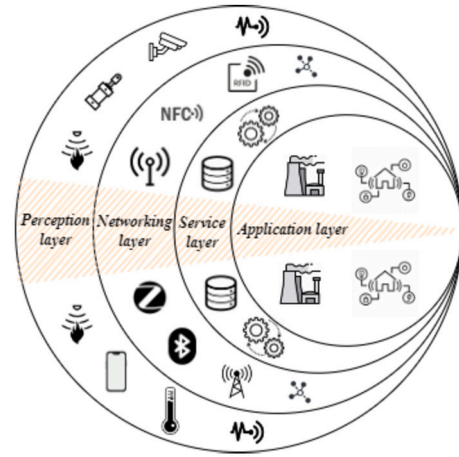


Fig. 2. IoT architecture.

necessitate a trade-off between decentralization, efficiency, and confidentiality.

2.2. Internet of things (IoT)

The IoT concept is established around the idea of enabling virtually anything to connect to the Internet and actively communicate meaningful information [20]. It facilitates the creation of intelligent systems by processing and analysing vast amounts of data collected from sensors and devices and actively responding to changing factors. IoT promises the creation of unprecedented business value with less human intervention, improved productivity, and effective decision-making. Technically, IoT is a heterogeneous network of interconnected devices, objects, and gadgets that are usually featured with capabilities such as sensing, actuation, computing resources, connectivity, and communication [20]. One can think of a wide range of applications that can benefit from the IoT concepts, including smart homes [21], smart cities [22], and smart grids [23].

Several proposed IoT architectures exist in the literature, such as Al-Fuqaha et al. [24] and Bouakouk et al. [25]. IoT architecture comprises four layers, mainly perception, networking, service and application, as shown in Fig. 2.

1. **Perception layer:** The perception layer is directly associated with the physical world. In this layer, sensors are used to collect environmental information, while actuators are used to respond to received data or changing factors [26].
2. **Networking layer:** The network layer is responsible for connecting things to the Internet to enable the transmission of their collected data from the perception layer to their destination. There are various networks, connectivity, communication protocols, and technologies involved. Some of them are specifically suited for lightweight IoT devices (e.g., Message Queuing Telemetry Transport and Constrained Application Protocol), while others are well-established and not only used for IoT purposes but can be found elsewhere. For instance, well-established connectivity protocols and technologies in the network layer can involve wireless networks (e.g., Wi-Fi and Bluetooth), Wired networks (e.g., Ethernet), and Cellular networks (e.g., 3G, 4G, or 5G).
3. **Service layer:** The service layer is responsible for processing and managing the data collected from the perception layer and transmitted through the network layer. This layer handles various tasks, such as things management, data storage, data analysis, and decision-making. Moreover, the service layer is also concerned with business logic control, configuration, updates, monitoring, and security [26].
4. **Application layer:** This layer represents the value generated by connected things on the Internet. The generated value can take the form of an emerging application or an enhanced user experience. Applications can be divided into two categories: critical applications (e.g., industrial) or quality-of-life applications (e.g., smart home). In either case, the application layer interfaces with end users and other machines for interaction and communication with the underlying IoT ecosystem.

2.3. Motivation for integrating blockchain with IoT

Several challenges associated with typical IoT systems call for revisiting current models and incorporating revolutionary and effective technologies and methods. When considering the features of blockchain technologies, they present an appealing opportunity to utilize blockchain technology to overcome challenges of IoT or simply for creativity and innovation grounds [27]. The following are some IoT challenges that can potentially be addressed by blockchain features:

1. **High availability:** It is a vital aspect for critical IoT systems, such as manufacturing and smart grids, to remain functional whenever needed. Any downtime or interruption in the availability of any part (e.g., sensors, devices, networks, servers, and applications) of such heterogeneous and complex IoT systems can hinder effective operations and lead to significant life and financial losses. Most traditional IoT systems are heavily dependent on centralized servers, in which a failure at the latter causes the entire ecosystem to degrade or, worse, completely get out of service [28]. One of the essential advantages of blockchain is its ability to mitigate the risk of a single failure point, which sounds appealing for resolving such issues in IoT systems [29]. In blockchain, all validating nodes can continue to validate transactions and record their outcomes, even if some nodes are out of service. Once the absent nodes are available, they can synchronize their ledger and carry on their operations from that point.
2. **Trust and accountability:** A typical IoT application involves multiple stakeholders and participants collaborating to achieve its intended objectives [30]. For instance, an IoT-based shipment tracking system may encompass suppliers, international customs authorities, cargo companies, warehouses, and delivery service providers. A key challenge in such systems is determining a trusted entity responsible for transaction validation and ensuring data integrity. While independent third-party entities may appear to be a viable

solution, they present several drawbacks, including high operational costs, a single point of failure, and potential misconduct [31]. Blockchain technology addresses these concerns by eliminating the need for a central authority or intermediary. It enhances trust through a consensus mechanism, wherein the majority of participants must validate transactions based on predefined protocols and rules [29]. Furthermore, the inherent properties of blockchain, namely transparency, traceability, and immutability, serve as an accountability mechanism.

3. **Security and resilience:** IoT applications depend on data for decision-making and the intended business value. Hence, it is important to maintain data integrity and secure IoT systems from cyber threats. Each IoT node can present a potential failure point when exposed to cyberattacks such as distributed denial of service attacks, injection attacks, and other security threats [32]. Additionally, the reliance on centralized systems, such as a single cloud provider or supervisory control and data acquisition systems, can introduce a single point of failure, when being vulnerable to attacks [33].

Blockchain is infamous for its security-by-design feature, employing well-established cryptography algorithms and hashing techniques wherever possible in the journey of transactions [34]. To begin with, the Zero-trust architecture is adopted in the vast majority of blockchain platforms [35]. A public key infrastructure (PKI) is also essential for sending, processing, and persisting transactions in these blockchain platforms. The data organization into a chain of hashed blocks and transactions consolidates immutability, making it difficult for any blockchain node to tamper with the data in their local ledgers maliciously. Even if a node manages to execute malicious actions or is compromised for any reason, several measures are in place to tackle the issue, such as the distribution nature of the blockchain network, the mitigation of points of a single failure, replication and consensus mechanisms. These are some of the reasons why blockchain can be considered for securing IoT networks and preserving data integrity.

2.4. Challenges associated with the convergence between blockchain and IoT

Despite the opportunities of integrating blockchain with IoT, several challenges and points of consideration must be accounted for when designing a blockchain-based IoT solution, which arises from the complexities of both technologies and the difficulties in implementation. The following are examples of:

1. **Resource constraints:** Many IoT devices are often battery-powered and have limited energy resources that make energy efficiency a critical factor in the design of blockchain-based IoT architectures [8]. On the other hand, several blockchain platforms employ heavyweight consensus protocols (i.e., proof of work (PoW)) and security measures (i.e., PKI and zero knowledge proof), which require high computational resources to cope with. Furthermore, several IoT devices are limited in their capabilities, such as processing, storage, connectivity and communication protocols [36]. Consider a scenario where IoT devices must continuously operate within the blockchain network, which calls for the need to address associated challenges, given the lack of sufficient IoT capabilities.
2. **Scalability:** Many IoT applications can generate a massive amount of datasets from an enormous number of IoT devices, such as in healthcare and the power grid. Unlike centralized systems, scalability is a major challenge in integrating blockchain with IoT, given the level of distribution and decentralization of most blockchain platforms. This is particularly imminent with blockchain platforms that depend on heavyweight consensus mechanisms (e.g., PoW), which severely impact the overall performance and limit transaction throughput [8]. This fact defeats the purpose of utilizing

blockchain to reduce latency in the overall system, even if it succeeds in reducing latency between IoT devices and the local blockchain nodes.

3. **Integration complexity:** Both IoT and Blockchain are complex and heterogeneous, each with its own set of protocols, standards, and architectures. This complexity presents several challenges that can hinder research and development efforts in experimenting with blockchain-based IoT solutions [37]. While Blockchain platforms are mostly accessible thanks to their open-source nature and scalable cloud services, a significant obstacle lies in accessing sufficiently actual IoT infrastructure for experimental purposes. Implementing a real-world IoT system to assess the viability and effectiveness of a blockchain-driven approach can be laborious and demanding in terms of resources, necessitating the setup of the tangible quantity of equipment, acceptable infrastructure scale, and the combination of different elements and technologies.

2.5. Performance evaluation approaches

Evaluating the performance of Blockchain-based IoT systems is essential for assessing their viability and effectiveness. Performance evaluation may take place in the early stages or after the production stage. Given the complexity of both such systems and the difficulty of accessing a large and sufficient IoT infrastructure. The performance evaluation must account for the unique characteristics and requirements of each of the underlying technologies and the convergence between them [38]. The distributed nature of blockchain networks, combined with the heterogeneity and resource constraints of IoT devices, necessitates the design of experiments that can capture the complex interactions and performance trade-offs between the various components. This involves the selection of appropriate metrics, such as transaction throughput, latency, and energy consumption, as well as the design of realistic workloads and network properties that can practically reflect real-world deployments.

2.5.1. Simulation approach

Experimenting with real-world systems can often be impossible, impractical, or costly [39]. Simulation is a widely used method to mimic system behavior and evaluate performance in a controlled and reproducible manner [40]. It involves creating an abstract model of the system using mathematical formulas, structural relationships, and logical relationships. Models can be classified as deterministic, stochastic, static, or dynamic, with dynamic models further divided into continuous or discrete [41,42].

Simulation helps predict system behavior under various conditions, test new policies, and explore design alternatives. Its modularity and configurability make it a practical tool for performance evaluation [39]. Therefore, researchers use IoT simulators to model and test systems in controlled environments, avoiding the expense and complexity of physical deployments [43].

Numerous research efforts have been dedicated to simulating diverse IoT aspects, including cloud environments (e.g., CloudSim [44], SimIoT [9]), fog/edge computing (e.g. iFogSim [45], EdgeCloudSim [11], Edge-Fog [10], IoTsim-Edge [46], EdgeNetworkCloudSim [47], Myi-FogSim [48], FogNetSim++ [49], IoTsim-Osmosis [12]), and application-layer interactions (e.g., IoTsim [9]).

2.5.2. Middleware approach

The concept of Middleware plays a crucial role in complex systems by serving as an intermediary between disparate components and bridging the gap between them [50]. The concept of middleware abstracts the complexity of different technologies, platforms, and protocols, which allows developers to focus on business logic rather than delving into unnecessary concerns associated with integration and compatibility. The middleware can manage and facilitate seamless interactions among diverse software applications or components, often across distributed net-

works [51]. It also incorporates features that improve scalability and performance, such as load balancing, caching, and failover mechanisms, while ensuring robust security measures, including authentication, authorization, and encryption. In environments characterized by complex transactional processes, such as IoT and Blockchain, the middleware concept is indispensable to maintain data consistency and integrity [50].

Despite research efforts in developing IoT simulators, the lack of direct interoperability with blockchain platforms hinders the realistic evaluation of blockchain-based IoT scenarios. This limitation creates a significant gap in testing and benchmarking, making it challenging to assess blockchain-enabled IoT solutions' performance, security, and scalability under realistic conditions. As a result, researchers and developers face difficulties in validating IoT-blockchain interactions, optimizing system performance, and identifying potential bottlenecks before real-world deployment. To overcome these challenges, a middleware solution facilitates the seamless integration of IoT simulators with blockchain platforms for performance evaluation and realistic benchmarking of blockchain-based IoT systems. As illustrated in Fig. 3, designing such middleware involves overcoming key challenges arising from divergent operational paradigms, technical constraints, and architectural differences. IoT simulators primarily use discrete-event modeling, processing system events sequentially over a virtual timeline. In contrast, blockchain platforms function as continuously operating, decentralized ledgers, making real-time synchronization complex. Additionally, IoT devices generate high-frequency, asynchronous data streams, which conflict with blockchain's inherent transaction latency and block interval delays. These discrepancies introduce challenges in ensuring efficient data consistency, throughput, and reliability in the integrated IoT-blockchain environments.

3. SimBlockLink: a middleware for linking IoT simulations with real blockchain platforms for enhanced performance evaluation

We aim to utilize IoT simulators as workload generators for evaluating real-world blockchain platforms based on simulated IoT models. Therefore, as Fig. 4 illustrates, this paper suggests the utilization of a middleware solution for integrating two distinctive environments, which are a real blockchain platform (namely Hyperledger Fabric [14]) and an IoT simulator (namely IoTsim-Osmosis [12]). See section 6 for the rationale behind this selection.

However, the challenge of bridging the gap between executing distributed environments (such as Hyperledger Fabric) and simulated environments in IoT simulators (such as IoTsim-Osmosis) continues to exist. The integration challenges are not limited to the IoT simulator's ability to communicate and interact with the blockchain platform, but also extend to the distinctive nature of both sides. Several existing IoT simulators are concerned with a discrete event representation throughout a virtually short-lived time. These simulators rely on predefined calculations and execution duration to generate their output.

Consequently, a naive integration between these two disparate and heterogeneous execution environments fails to align IoT simulators with the real blockchain platform. An example of a naive integration is the tight coupling of the IoT simulator with the blockchain platform, which will negatively influence the execution run-time of IoTsim-Osmosis. Consider that the IoT simulator must be stopped whenever it interacts with the blockchain. As a result, the IoT simulator, *being a discrete event simulator*, will demonstrate miscalculations, resulting in unrealistic outputs. Thus, the middleware must take into account the distinctive nature between IoT simulation and real blockchain platforms to ensure accurate performance evaluation and benchmarking.

3.1. Potential real-world applications of SimBlockLink

SimBlockLink offers a middleware solution allowing IoT simulators to be connected with actual blockchain systems, therefore enabling

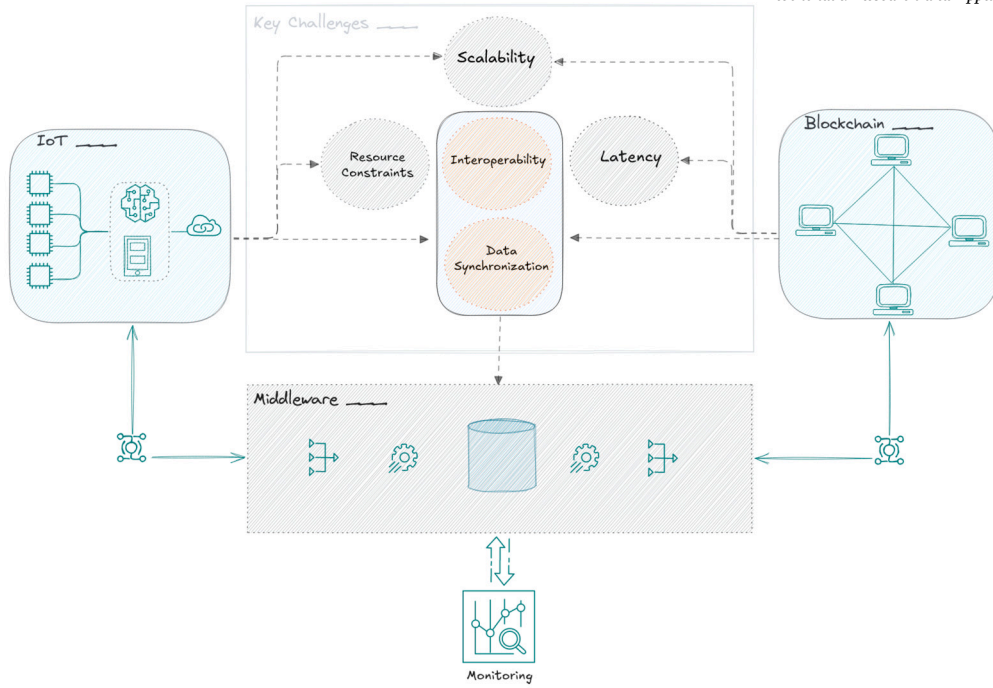


Fig. 3. Key challenges of integrating blockchain and IoT.

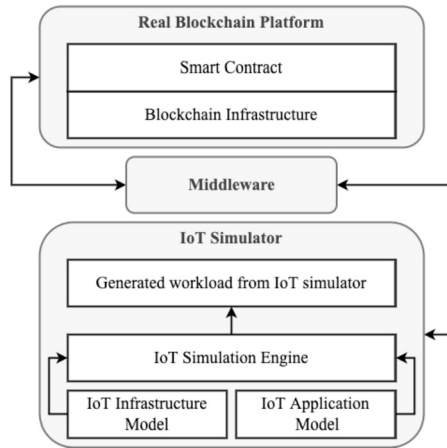


Fig. 4. Middleware to integrate IoT simulator with a real blockchain platform.

benchmarking and performance evaluation in controlled but scalable settings. This feature makes it relevant in many real-world fields where blockchain technology is under increasing discussion as a means of improving security, openness, and data integrity. One such area is smart healthcare, where SimBlockLink can be used to assess wearable health devices and share blockchain-based patient data between hospitals and medical research institutes. For electronic health records and remote patient monitoring systems, especially blockchain integration, guarantees safe, auditable, and privacy-preserving data sharing. In intelligent transportation systems, the middleware can help to simulate vehicle data logging and safe communication between linked and autonomous cars (CAVs). SimBlockLink lets researchers evaluate whether distributed ledgers could be used for accident forensics, congestion management, and vehicle-to-anything (V2X) communication by enabling blockchain-based data provenance. Another relevant use is in energy management and smart grids, where SimBlockLink can help to replicate peer-to-peer energy trading. Decentralized energy markets, including IoT-enabled solar panels, wind farms, and energy storage systems, need reliable transaction platforms to enable safe, effective,

open energy trades among consumers. Beyond these examples, SimBlockLink is adaptable to any large-scale IoT application that necessitates trust, auditability, and decentralized control. By providing a means to evaluate blockchain performance under realistic IoT workloads, SimBlockLink serves as a valuable tool for researchers and industry practitioners exploring blockchain adoption in domains where data security, interoperability, and performance optimization are of critical importance.

3.2. SimBlockLink: proposed middleware architecture

This paper proposes a middleware architecture, namely: SimBlockLink, for bridging the gap between IoT simulations and real blockchain platforms. As illustrated in Fig. 5, IoTsim-Osmosis is selected as an IoT simulator, while Hyperledger Fabric is selected as a real blockchain platform. The main functionality of SimBlockLink is to provide IoT simulators with the ability to do the following:

- Act as a workload generator to benchmark and evaluate the responsive blockchain platform.
- Transact with the respective blockchain platform, which requires authentication, connectivity, communication, and transactions fail-safe mechanism.

The middleware architecture adopts the separation of concerns principle to address the distinction between the IoT simulator and the blockchain platform. The middleware architecture manifests the separation of concerns by introducing two separate but collaborative entities: *Agents* and *Workers*. While each interfaces with a distinctive environment, they interact with each other through concurrent storage. The following describes each of the agents and workers.

- **Agents** are designed to consider the nature of the simulated environment. Hence, they interface with the simulator to gather data from the simulated IoT environment and store them in concurrent storage. The behaviour of these agents, such as how frequently and periodically data are gathered, is not specifically mandated and can be instructed as required by the middleware manager.

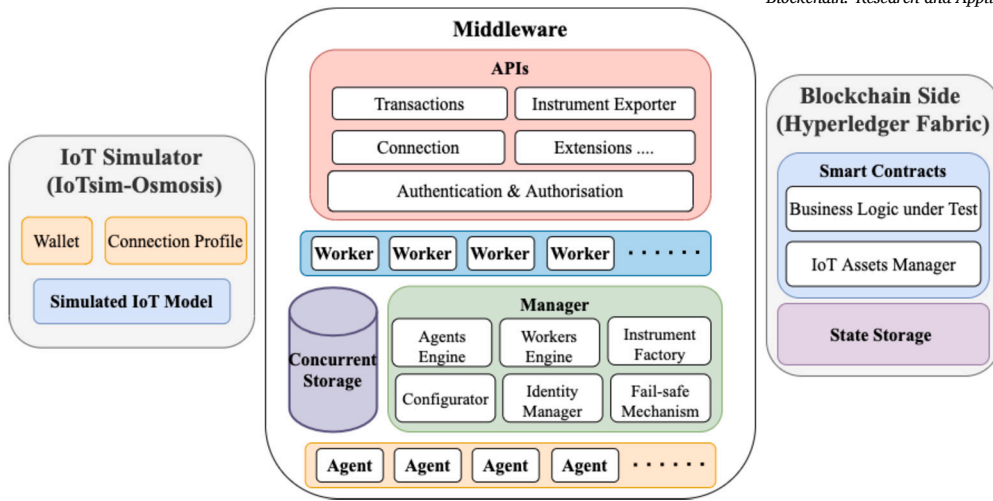


Fig. 5. SimBlockLink: a middleware for integrating IoT Simulators with blockchain platforms.

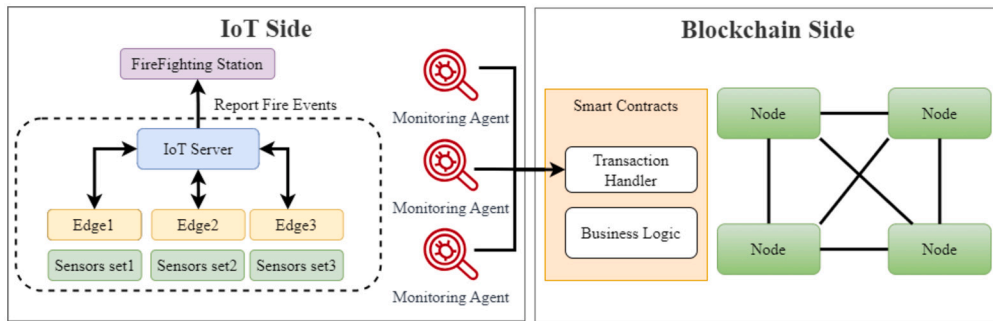


Fig. 6. Simulated IoT firefighting case study using SimBlockLink for blockchain integration.

- **Workers** are designed to handle the nature of the blockchain platform. Therefore, they are mainly responsible for bundling the data gathered by agents from the concurrent storage and submitting them as transactions to the blockchain. Therefore, the middleware provides a set of application programming interfaces (APIs) that workers can leverage when transacting with blockchain. For example, workers must be authenticated and connected to the blockchain platform to submit transactions.

4. Implementation

4.1. Smart contracts

From the blockchain side, the middleware interfaces with various smart contracts that fall into two main categories, as follows:

- **IoT asset manager:** Smart contracts dedicated to representing, defining and managing IoT assets and their properties. For instance, in light of the IoT-based firefighting scenario, an asset can be a home with properties such as owner, address, and location. SimBlockLink utilizes the IoT asset manager to execute create, read, update, and delete operations (CRUD) methods at the blockchain state storage to manage IoT assets.
- **Business logic under test:** Smart contracts that represent the business logic under test. For instance, the scenario presented in Fig. 6 assumes fire alerts are submitted from IoT sensors through monitoring agents to the blockchain platform for processing and further actions. Therefore, the middleware interfaces with associated smart contracts and stresses them with the required workload from the IoT simulator in the form of submitted transactions.

4.2. Thresholds specifier

The middleware considers a specifier that enables defining a set of thresholds against metrics generated by the IoT simulator. The threshold specifier is an essential part of the *configurator* component provided by the middleware manager. The threshold specifier supports basic comparative operators against a specific value, such as greater than, less than, equal to, not equal to, and not. For example, assume a threshold requirement v_i to be *fireAlert* $\neq$ *false*. Therefore, the *thresholds specifier* enables filtering the generated metrics accordingly. There can be several usages for such a threshold requirement. For example, one can instruct agents to capture metrics only when a fire event occurs. Additionally, agents can be instructed to classify generated metrics as positive or negative based on this threshold. As illustrated in Algorithm 1, the middleware specifies a threshold requirement v_i . The middleware also determines whether v_i must persist at both sides, the simulator and the blockchain, when the flag indicates *true* $\rho = \text{true}$; otherwise, v_i only persists locally. Persisting v_i on the blockchain's state storage requires triggering the IoT assets manager smart contract. If the creation transaction succeeds, the middleware generates a duplicate of the threshold requirement v_i at the local storage; otherwise, it terminates the entire process.

4.3. Agents

The middleware provides agents that interface with the IoT simulator. Agents can be deployed wherever metrics $\{m_1, \dots, m_n \in M\}$ are generated from the underlying simulator. For instance, a generated metric m_i can be the status of an alarm sensor, which can be positive (fire event) or negative (no fire event). Agents are aware of specified thresholds, which are the ones created by Algorithm 1. Each agent is assigned

Algorithm 1 Threshold Specifier Creation.

Require: v_i $\triangleright$ Threshold requirement
Require: ρ $\triangleright$ whether v_i is persisted in both sides
Require: $maxRetry$ $\triangleright$ Max allowed attempts to submit transactions
Ensure: whether v_i and v'_i are successfully created.

```

1:  $\eta = 0$   $\triangleright$  attempts count to submit transactions
2: if  $\rho$  then  $\triangleright$  Blockchain transaction is required
3:    $\alpha \leftarrow false$   $\triangleright$  Transaction Status Flag
4:   repeat
5:     submit  $v_i$  to the blockchain side.
6:      $\eta \leftarrow \eta + 1$ 
7:     if  $v_i$  then  $\triangleright$  successfully created
8:        $\alpha \leftarrow true$ 
9:     end if
10:  until  $(\alpha = true) \vee (\eta \geq maxRetry)$ 
11: end if
12: if  $\alpha \vee \neg \rho$  then  $\triangleright$  successful Blockchain transaction or not required
13:   Create  $v'_i$  at the simulator-side
14: end if

```

a set of generated metrics $\{m_i \in M\}$. Multiple agents can be deployed at the IoT simulator level to capture their assigned metrics once produced by the IoT simulator. Agents instantly evaluate captured metrics against a specified threshold $\{v_i \in Y\}$. Based on the evaluation, agents report the results indicating whether a generated metric is classified as a breach (B) or in compliance (C). Algorithm 2 illustrates the process of examining generated metrics. For simplicity, the algorithm conveniently uses the increment notion ($C++$ or $B++$) to express the ability of agents to classify generated metrics and store them accordingly.

Algorithm 2 Agents' Evaluation Behaviour.

Require: v_i and m_i $\triangleright$ Threshold requirement and generated Metric
Ensure: C or B . $\triangleright$ compliance or Breach

```

1:  $l \leftarrow v_i(level)$   $\triangleright$  Threshold level (e.g. GreaterThan, LessThan, Equals, Not)
2:  $v \leftarrow v_i(threshold)$   $\triangleright$  get threshold value
3: if  $l = GreaterThan$  then
4:   if  $m_i < v$  then
5:      $B++$ 
6:   else
7:      $C++$ 
8:   end if
9: else if  $l = LessThan$  then
10:  if  $m_i > v$  then
11:     $B++$ 
12:  else
13:     $C++$ 
14:  end if
15: else if  $l = Equals$  then
16:  if  $m_i \neq v$  then
17:     $B++$ 
18:  else
19:     $C++$ 
20:  end if
21: else if  $l = Not$  then
22:  if  $m_i = v$  then
23:     $B++$ 
24:  else
25:     $C++$ 
26:  end if
27: else
28:   undefined
29: end if

```

The middleware recognizes the fact that processing blockchain transactions introduces an unnecessary delay to the simulator's execution runtime. Therefore, it considers the adverse effect of the blockchain transaction life cycle on the performance and reliability of the underlying IoT simulator. While agents are loosely coupled to the IoT simulator,

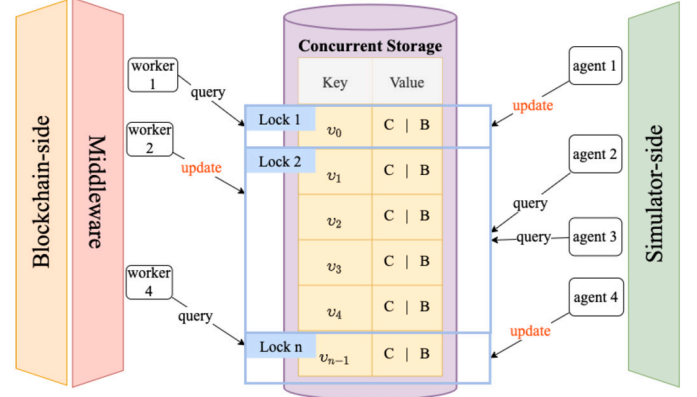


Fig. 7. The locking mechanism on the concurrent storage for agents or workers.

they are completely decoupled from the blockchain platform. Consequently, agents interface with the local concurrent storage instead of establishing direct communication with the associated blockchain platform. The concurrent storage maintains a set of specified threshold requirements $\{v_i \in Y\}$ along with their indicator sets, which are B set for breach instances to v_i or C set for compliance instances with v_i . See Fig. 7 for a visual elaboration.

4.4. Workers

For blockchain benchmark and evaluation purposes, the middleware dedicates and schedules a pool of concurrent entities called workers. Unlike agents, these workers interact and transact with the blockchain side. The main worker's role is to stress load the smart contract under test. Recall that the local storage is concurrently shared between agents and workers (See Fig. 7). Therefore, workers repeatedly examine the local storage to check whether there is an update by agents on the C and B sets associated with every threshold requirement $\{v_i \in Y\}$. If any workers attempt to submit the latest update to the respective smart contract, they will update the status of both the C and B sets in the state storage. This update reflects the difference between the workers' most recent values and their reported values.

Algorithm 3 offers a detailed insight into the operational dynamics of workers. Initially, the middleware sets up a worker with specific attributes: (i) the designation of a smart contract under test, (ii) a particular method (or function) within that smart contract designated for transaction execution, such as "report fire event," and (iii) a set of threshold criteria $\{v_i \in Y\}$. Workers refrain from transacting with the blockchain unless there is a status change in the concurrent storage concerning their assigned v_i . This is important to mitigate the unnecessary load on the blockchain infrastructure that might cause undesired miscalculations. As stipulated in Algorithm 3 Line 4, workers proceed with submitting B or C data to the blockchain only when $B > 0$ or $C > 0$. When the transaction is successful, the worker reflects that on the B and C values at the concurrent storage.

4.5. Storage concurrent locking mechanism

Workers reflect successful transactions by updating B or C sets, which is not trivial due to the operations overlap and racing between agents and workers on these shared sets. Agents continuously update these datasets with new metrics collected from the IoT simulator, while workers aim to denote submitted datasets as being successfully submitted to the blockchain. Specifically, a significant delay occurs when a worker evaluates B or C sets from t_1 to the point it concludes a transaction with the blockchain at t_2 . During this interval, it's probable that agents or other workers have altered the state of B or C datasets. To mitigate potential race conditions and ensure data integrity, the mid-

Algorithm 3 Workers Behaviour.

Require: $\{contract, method, Q\}$
Ensure: Consistent update to B and C sets for assigned $\{v_i \in Y\}$

```

1: for each  $\{v_i \in Y\}$  do
2:    $B_{i1} \leftarrow B$  ▷ current state of Breach Set
3:    $C_{i1} \leftarrow C$  ▷ current state of Compliance Set
4:   if  $B_{i1} > 0$  or  $C_{i1} > 0$  then
5:     send Transaction (contract, method,  $v_i$ ,  $B_{i1}$ ,  $C_{i1}$ ) ▷ submit metrics to Blockchain
6:     if  $\alpha = true$  then ▷ Successful Transaction
7:        $B_{i2} \leftarrow B$  ▷ latest state of Breach Set
8:        $C_{i2} \leftarrow C$  ▷ latest state of Compliance Set
9:        $\Delta B \leftarrow B_{i2} - B_{i1}$  ▷ Difference between latest and current Breach Set
10:       $\Delta C \leftarrow C_{i2} - C_{i1}$  ▷ Difference between latest and current Compliance Set
11:      if  $\Delta B \neq 0$  then
12:         $B \leftarrow \Delta B$  ▷ Apply difference and update Breach Set
13:      else
14:         $B \leftarrow 0$ 
15:      end if
16:      if  $\Delta C \neq 0$  then
17:         $C \leftarrow \Delta C$  ▷ Apply difference and update Compliance Set
18:      else
19:         $C \leftarrow 0$ 
20:      end if
21:    end if
22:  end if
23: end for

```

middleware employs a concurrent locking mechanism designed to handle concurrent modifications effectively:

- Only one entity, whether an agent or worker, is granted the authority to modify (update) any C or B dataset at any given time.
- During the period $\Delta t = t_2 - t_1$, agents are allowed to update B or C datasets with new metrics.
- A worker, after finalizing a transaction on the blockchain, is restricted from updating the datasets associated with $\{v_i \in Y\}$ until ensuring no other entity currently possesses the lock. Once the lock is available, the worker can secure it to incorporate the changes ΔB and ΔC , thus marking them as reported and avoiding redundant submissions.
- To maintain accuracy and prevent errors, workers refrain from making changes ΔB and ΔC to the storage if the values are negative. This indicates that there is no actual difference to be recorded.

The middleware organizes concurrent storage as key-value pairs, with the key representing a specific threshold requirement $\{v_i \in Y\}$ and the value encompassing the related metric datasets B and C . This storage system incorporates a locking mechanism that selectively authorizes write operations (including insertions, deletions, and updates) to a single entity at a time while permitting unrestricted read access to any entity. To optimize performance, the storage is segmented, allowing write locks to be applied on a per-segment basis rather than across the entire storage space. This segmentation facilitates simultaneous write operations by multiple entities on different parts of the storage, thus enhancing throughput and overall system efficiency. Fig. 7 illustrates an example of how storage segmentation and locking mechanisms operate, showcasing interactions between agents and workers:

1. In Segment 1, agent 1 secures a lock to input data related to breaches (B) and compliances (C) for metrics compared against v_0 . Worker 1, meanwhile, can review the latest data for v_0 without hindrance, as the lock does not restrict read operations.
2. Segment 2 is locked by worker 2 following a successful metric transaction concerning a group of threshold requirements $\{v_1, v_2, v_3, v_4\}$, intending to update ΔB and ΔC accordingly. Agents 2 and 3 can

still access data within this segment for query purposes, avoiding operational conflict.

3. A worker or agent needs to perform a write operation on a specific segment, it must wait for the existing lock to be released before acquiring the lock for its use.

4.6. Smart contract benchmarking functionality

The middleware incorporates a built-in benchmarking tool for assessing blockchain performance. It monitors fundamental performance indicators for each worker's transaction, including throughput, latency, success, and failure rates. The benchmarking mechanism follows the performance metrics assessment guidelines proposed by the Hyperledger Performance Working Group [52]. The subsequent sections delineate the processes involved in the indicators' instrumentation, data collection, exportation, analytical evaluation, and graphical presentation.

Fig. 8 depicts assembling and exporting performance metrics for each smart contract method invoked by workers. A dedicated module, termed *Instrument Exporter*, leverages Micrometer²—an open-source, vendor-neutral tool that integrates with a broad spectrum of monitoring solutions. The *instrument exporter* generates a set of metrics used by the middleware to measure the performance of the smart contract under test. These metrics include successful transactions (T_s), failed transactions (T_f), and transaction latency (T_d).

Algorithm 4 Instruments Composition.

Require: $\{contract\}$
Output: $\{T_s, T_f, T_d\}$

```

1: let  $I = \{method_i \mid i \in \mathbb{N}\}$  ▷ List of all method of the smart contract
2: for each smart contract invocation do
3:   if  $method \notin I$  then
4:     compose Instruments  $\{T_s, T_f, T_d\} \in method$ 
5:     include  $method$  into  $I$ 
6:   end if
7:    $timer_{start}$ 
8:   submit Transaction to  $method$ 
9:   if Transaction is successful then
10:     $timer_{end}$ 
11:     $T_d = timer_{end} - timer_{start}$ 
12:     $T_s++$ 
13:  else
14:     $T_f++$ 
15:  end if
16:  publish  $\{T_s, T_f, T_d\}$  to HTTP server
17: end for

```

As outlined in Algorithm 4, the middleware avoids repeated instrumentation by verifying whether the smart contract has been instrumented. If not, the *instrument exporter* initiates a fresh set of metrics, including T_s for successes, T_f for failures, and T_d for latency measurements. For each transaction a worker initiates, the middleware updates either T_s or T_f accordingly. The transaction latency T_d is determined by the elapsed time, calculated (1)

$$T_d = timer_{end} - timer_{start} \quad (1)$$

where $timer_{start}$ marks the transaction's submission to the blockchain and $timer_{end}$ the moment the transaction completion. It's important to note that the calculation of T_d intentionally omits unsuccessful transactions due to the irrelevance of the latency indicator.

The middleware orchestrates the integration of three key components to collect and visualize the performance indicators $\{T_s, T_f, T_d\}$,

² <https://micrometer.io>.

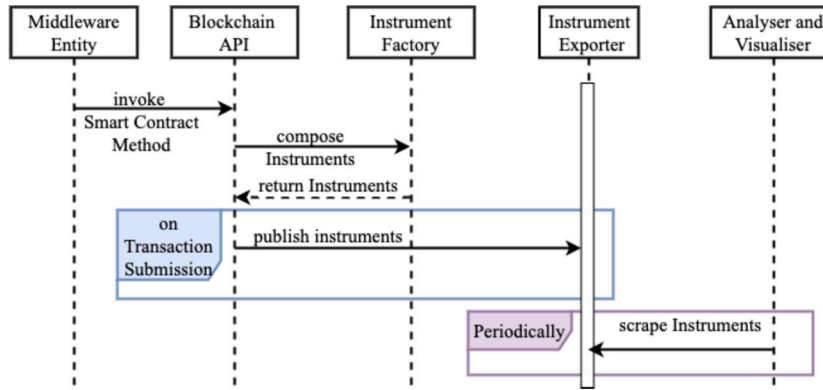


Fig. 8. Process of assembling and exporting metrics.

which are: (i) An HTTP server tasked with the export of metrics for each smart contract's method, (ii) Prometheus [53] for periodic collection of the indicators from the HTTP server, and (iii) Grafana [54] to compile the benchmarking data and facilitate their visualization. Accordingly, a series of benchmarking parameters can be established to evaluate the smart contract's performance as delineated in the following equations:

$$Avg_{tps} = \frac{\sum_i^n T_s}{\sigma} \quad (2)$$

$$Avg_{latency} = \frac{\sum_i^n T_d}{\sum_i^n T_s} \quad (3)$$

$$s_{rate} = \frac{T_s}{T_s + T_f} \times 100 \quad (4)$$

$$f_{rate} = \frac{T_f}{T_s + T_f} \times 100 \quad (5)$$

Avg_{tps} quantifies the transaction throughput, defined as the sum of successful transactions over σ . σ is defined as $lct - fst$, where fst is the timestamp of the first transaction that was successfully processed, and lct is the timestamp of the final transaction that was committed to the blockchain ledger. s_{rate} and f_{rate} represent the fractions of transactions that were successful and failed, respectively, out of the total number of transactions. The average latency $Avg_{latency}$ is determined by aggregating the total time taken to execute each transaction and dividing this by the number of successful transactions.

4.7. Middleware manager

The middleware manager is the main point for configuring, managing, and controlling the middleware.

4.7.1. Agents and workers

The manager controls and manages the behavior of both agents and workers as per explained in Section 4.3 and Section 4.4; respectively. The manager enables users to specify any configurable parameters of agents or workers. For instance, assign a set of IoT metrics to agents or assign a smart contract to be load-tested by workers.

4.7.2. Identity management

Hyperledger Fabric functions as a permission blockchain platform. Thus, a cryptographic wallet is instrumental in connecting and facilitating communication between the middleware manager, workers, and the blockchain network. For instance, it allows workers to use their specific identities to compose and endorse transactions during smart contract executions. As depicted in Fig. 9, the wallet creation requires registration and enrollment to the blockchain platform. The initial step involves registering a middleware's admin identity, which produces an ID and a secret key. Then, the middleware presents these credentials to a certificate authority (CA) on the blockchain part for self-enrollment. The

CA assesses these credentials and assigns an admin identity to the middleware, which features administrative capabilities that facilitate the registration and enrollment of workers onto the blockchain network. The Identity within this context combines public/private key pairs encapsulated within a digitally signed X.509 certificate. The admin can then use their identity to generate another identity sufficient for workers' purposes.

4.7.3. Transactions fail-safe mechanism

To ensure the integrity of the evaluation results, the middleware incorporates a fail-safe mechanism for handling possible transaction failures. As illustrated in Fig. 10, the middleware assigns a maximum number of attempts (e.g., $maxRetry \leftarrow 5$). This is particularly useful in scenarios where, despite multiple attempts, a transaction fails to be accurately processed due to external factors such as network or blockchain environment. The middleware closely monitors transaction events on the blockchain for every smart contract call. It verifies whether a transaction is successfully executed and recorded on the ledger. If a transaction fails, the middleware attempts resubmission until the maximum number of attempts is reached. This aids in making informed decisions regarding the experiment's validity, thus avoiding possible deviations from the anticipated behavior.

4.7.4. APIs

The middleware interfaces with the distinctive components: the blockchain platform and the IoT simulation tool. It also provides performance metrics for monitoring and visualization tools. Given the distinction of these components, the middleware manager orchestrates communication and connectivity through APIs. For instance, it provides workers with APIs to connect, authenticate and transact with smart contracts. The middleware manager also provides performance metrics to monitoring tools such as Prometheus. It also provides metrics for visualization tools such as Grafana. The design of middleware also encourages users to extend existing APIs to support and satisfy various research needs by adopting the concept of APIs, which follows the separation of concerns principle.

4.8. Development environment

This paper contributes a reference implementation of the proposed middleware (SimBlockLink), which is publicly provided on GitHub.³

Given that Hyperledger Fabric provides a Java-based client software development kit (SDK), Java is selected as the main programming language for implementing the proposed middleware. Therefore, the implemented middleware can be natively imported to any Java-based

³ SimBlockLink Repository: <https://github.com/aakzubaidi/BMBmid-Middleware>.

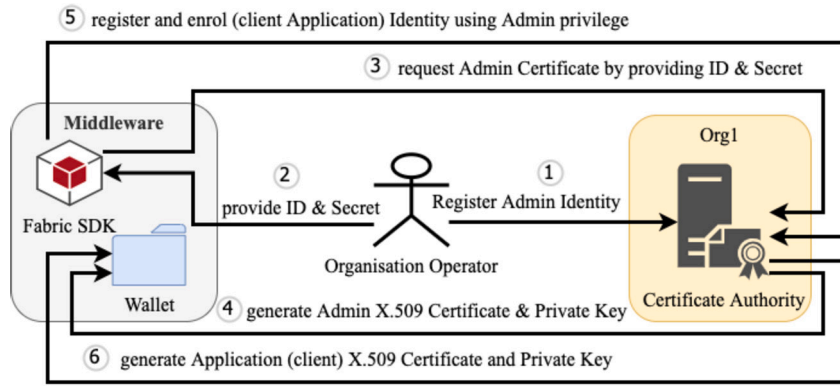


Fig. 9. Authenticating workers to the blockchain platform.

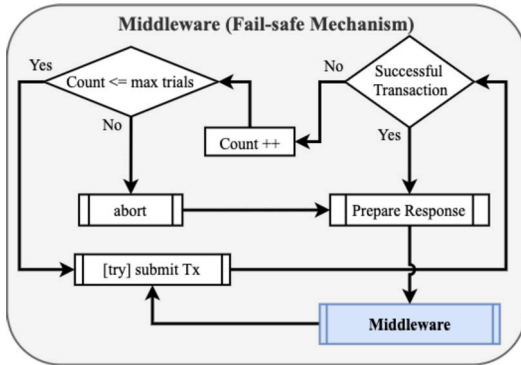


Fig. 10. A fail-safe mechanism for handling possible transaction failures.

simulator as a library. For other non-Java simulators, the middleware can be run as a server to intermediate between non-Java simulators and the blockchain platform through APIs.

5. Evaluation

This section presents an evaluation of SimBlockLink through the validation of its concurrent storage mechanism (Section 5.1), and a use case evaluation (Section 5.2).

5.1. Validate concurrent storage mechanism

The correctness of the concurrent storage mechanism is crucial for the overall performance and reliability of the proposed SimBlockLink. Therefore, we designed a controlled experiment, as detailed in Algorithm 5, to validate its functionality and correctness. Table 2 illustrates the configuration settings for conducting the controlled experiment. Given the concurrent nature of the shared storage and the overlapped operations between agents and workers, the experiment focuses on the storage's ability to correctly store generated metrics against a predefined threshold (v). For each experiment iteration, agents (a_i) evaluate a set of metrics with increasing size from $|M| = 10^2$ to $|M| = 10^6$ (x ranges from 2 to 6). Metrics are categorized as compliance or breach based on their relation to the threshold. Therefore, agents update the concurrent storage accordingly as defined in Algorithm 2. Workers (w_i) process existing data in sets B and C and apply necessary updates. Note that there are delays introduced to control the experiment with regard to agents and workers. Agents initially wait 1 second before commencing their tasks. Then, they gather metrics from the simulator whenever generated. On the other hand, workers initially wait for 2 seconds and then periodically wait for 3 seconds to emulate the delay caused by the blockchain transaction processing. During the waiting period of a

Table 2

Test-bed configuration for concurrent storage validation.

Parameter	Quantity	Initial delay ($x = 2$ only)	Periodic delay
Agents	1	1 second	0 seconds
Workers	3	2 seconds	3 seconds
Generated metrics	10^x ($x \in \{2, 3, 4, 5, 6\}$)	N/A	N/A
Compliance rate	50%	50%	50%
Breach rate	50%	50%	50%

worker, agents are expected to gather as many generated metrics and store them in the concurrent storage.

The experiment's success relies on several criteria:

- **Agent evaluation:** Each agent (a_i) must evaluate all metrics ($tma = 10^x$) and correctly update sets B and C in the storage.
- **Worker processing:** Workers (w_i) must process all generated metrics ($tmw = 10^x$) without redundancy.
- **Classification accuracy:** The number of "compliance" C and "breach" B entries in the storage should reflect the actual metric distribution (50% each).
- **Synchronization:** The difference between agent-processed metrics (tma) and worker-processed metrics (tmw) should be zero ($\alpha = 0$) upon completion of each iteration, indicating proper synchronization.

Fig. 11 showcases the validation for the first iteration (where $x = 2$) with 100 generated metrics. The experiment successfully demonstrates SimBlockLink's ability to handle the workload properly.

5.2. Use case evaluation

Assume a scenario, illustrated in Fig. 6, where a firefighting station leverages IoT technology to gather real-time fire status data from connected smart homes, enabling faster response times and effective fire severity mitigation. Smart homes are equipped with fire detection sensors that autonomously detect fire incidents and transmit alerts to fire stations via the Internet. To ensure a secure and trustworthy messaging infrastructure, the fire station authority adopts blockchain technology, leveraging its immutability and decentralization to facilitate reliable communication between the station and thousands of connected homes while maintaining its reputation as a dependable emergency service provider.

To assess the performance of such a scenario, assume a large number (i.e. 30,000) of smart homes are connected to a blockchain platform and report their fire status (fire or no fire) to the respective smart contract. The large scale of the IoT infrastructure is challenging to access for research and development purposes. IoT simulators such as IoT-Sim-Osmosis can be a viable option given that SimBlockLink can connect

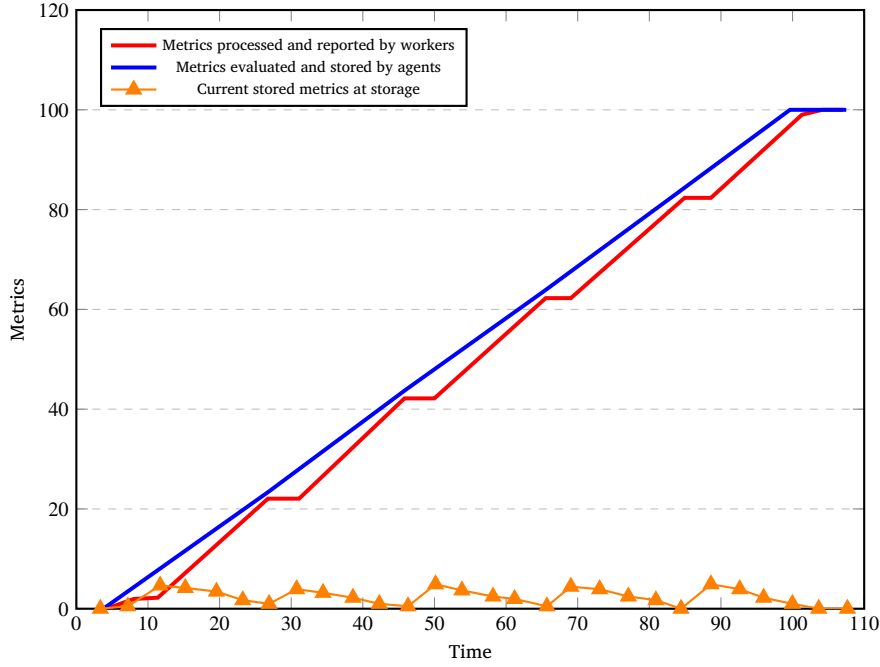


Fig. 11. Demonstrating correct operations on the concurrent storage by using 100 generated metrics.

it to a real blockchain platform and synchronize between a simulated model and a real blockchain platform.

This 30,000-device setting reflects a city-scale deployment and is treated throughout the paper as a stress-test workload, chosen deliberately to examine SimBlockLink's ability to sustain high transaction volumes on a live Fabric network. The following details the incorporation of SimBlockLink with the IoT-Sim-Osmosis simulator to evaluate the performance of the real blockchain platforms

5.2.1. Evaluation settings

- IoT simulation:** On the simulator side, we utilize the IoT-Sim-Osmosis simulator [12] for modeling the IoT-based firefighting architecture, which is depicted in Fig. 6. The modeled IoT architecture is organized into two primary layers: (i) three geographically distinct edge data centers, each serving different areas, and (ii) a network consisting of 30,000 smart houses organized into groups of 10,000 houses, with each group being connected to a designated edge data center. Table 3 highlights the identical specification of each IoT edge center. Each house has sensors that generate random fire statuses (positive or negative).
- SimBlockLink configurations:** SimBlockLink is treated as a library imported into the simulator's framework to connect the IoT simulator to the blockchain platform. The configuration of SimBlockLink involves key elements as follows:
 - The address (URI) of the certificate authority and its corresponding TLS certificate for secure communication.
 - Unique identification details (ID and secret) issued by the certificate authority for the simulator.
 - A specified location for storing generated identities within the simulator.
 - A connection profile for the blockchain network, allowing the simulator to discover and connect to relevant peers.
 - The name of the designated channel within the blockchain.
 - The name of the smart contract that is used for testing.
 - A threshold requirement v_i , which is set to $fireAlert \neq false$.

For preset expectations, Table 3 controls the experiment environment. Fig. 12 visualizes the workload flow from smart homes to the respective smart contract. There is an agent deployed for each of the three data centers, which are concerned with intercepting the fire

Table 3

Configurations and expectations.

Description	Configuration
Generated fire statuses	30,000, such that one fire status per smart home
Edge data centres	3 edge centres, each has:
	CPU: 4
	Bandwidth: 100 Mbps
	RAM: 4 GB
	MIPS/CPU: 250
	Storage: 200 GB
Agents allocated per edge data center	1
Fire status reporting frequency	1 fire-status record per 100 ms for each agent
Agent execution frequency	100 ms
Worker execution frequency (no initial delay)	1 s
Expected throughput (TPS)	1
Fire statuses updated per transaction	≈ 30
Expected total transaction number	≈ 1000

status of their associated smart homes. As in Algorithms 2, Agents are responsible for classifying the intercepted fire status based on the defined threshold v_i , and storing the outcome into the local concurrent storage. For consistency and predictability, the rate of generating a fire status for each smart home is throttled to one transaction every 100 ms. For consistency and predictability, each agent processes one fire-status record every 100 ms. On average, each agent injects into the concurrent storage a collection of 10 fire-status records, corresponding to 10 smart homes, every 1 s. Since there are three agents working simultaneously, there should be 30 records per second stored on the concurrent storage. As Algorithm 3 dictates, workers periodically examine the concurrent storage to observe whether there are new entries to submit to the blockchain platform. In this controlled experiment, the worker is configured to periodically inspect the concurrent storage every second. In light of the above, the worker should be able to submit approximately 1000 transactions in total to the blockchain platform with a send rate of 1 transaction per second, such that each transaction should report a collection of approximately 30 fire-status records, corresponding to

Algorithm 5 Controlled Experiment on Core Functionalities.**Require:** $\{M, a_i, w_i, v\}$

- ▷ M a set of metrics, such that $\{m_j \in M\} \subset \mathbb{Q}$
- ▷ a_i stands for an agent
- ▷ w_i stands for a worker
- ▷ v stands for a threshold requirement
- ▷ $tma \leftarrow 0$ Total metrics evaluated by a_i
- ▷ $tmw \leftarrow 0$ Total metrics processed by w_i

Ensure: valid state of concurrent storage and correct operations

```

1: Let  $v \leftarrow \text{Latency} \leq 1$ 
2:  $b \leftarrow 0; c \leftarrow 0$            ▷ counts for breach and compliance cases
3: Assign  $v$  to  $w_i$ 
4:  $x \leftarrow 100$ 
5: while  $x < 1000000$  do
6:    $j \leftarrow 0$ 
7:   while  $j < x$  do
8:     if  $x \leq 100$  then           ▷ Intentional delay for visualisation purposes
9:       Delay 1 second
10:    end if
11:    if  $(j \bmod 2) \leftarrow 0$  then
12:       $m_j \xleftarrow{R} \mathbb{Q} \mid m_j \leq \text{Latency}$            ▷ should be a compliance
13:    else
14:       $m_j \xleftarrow{R} \mathbb{Q} \mid m_j > \text{Latency}$            ▷ should be a breach
15:    end if
16:    Assign  $m_j$  to  $a_i$ 
17:     $j++$ 
18:  end while
19:   $\alpha \leftarrow tma - tmw$ 
20:   $\beta \leftarrow tma + tmw$ 
21:  if  $\alpha = 0 \wedge \beta = 2x$  then
22:    pass
23:  else
24:    fail
25:  end if
26:  if  $b \neq \frac{tma}{2}$  or  $c \neq \frac{tmw}{2}$  then
27:    incorrect classification (fail)
28:  else
29:    correct classification (pass)
30:  end if
31:   $x = x \times 10$ 
32: end while

```

approximately 30 smart homes. If the inspection of the blockchain ledger matches this expectation, we can generalize the outcomes to more transactions per second with different configurations.

3. **Blockchain deployment:** For this evaluation, we leveraged the Hyperledger Fabric v2.3.2 blockchain platform. The deployment utilized a cloud-based infrastructure provisioned with 32 vCPUs (Intel Xeon Gold 6140 processors clocked at 2.30 GHz) and 64 GB of RAM. These resources are equally distributed to 4 validating nodes and 5 orderer nodes. The consensus mechanism employed is Raft [55], a crash-fault tolerant mechanism recommended by the Hyperledger Community. A copy of a smart contract under test is deployed to each validating node. The smart contract mainly persists v_i as outlined in Algorithm 1, and receives transactions about the fire status of each home from an authenticated and authorized client.

5.2.2. Performance evaluation and analysis

The above stress-test workload was executed end-to-end, and Grafana was then used to visualize key performance indicators—throughput, latency and transaction success rate—revealing how both SimBlockLink and Hyperledger Fabric behave under sustained, high-volume conditions.

The key performance measurements are defined by Eqs. (2)–(5). The experiment produced 1030 transactions, surpassing the expected value of 1000 as indicated in Table 3. Fig. 13 shows a visualized output by SimBlockLink about the essential performance measurement of the

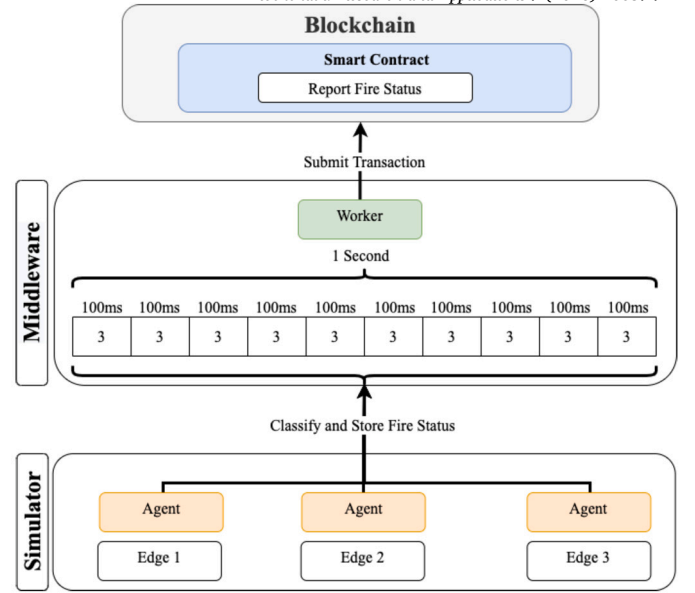


Fig. 12. Benchmarking workload flow from simulated edge centers to the smart contract.

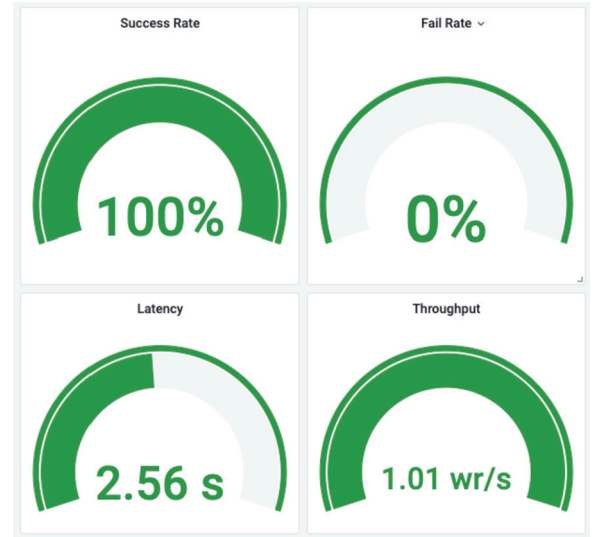


Fig. 13. Key Blockchain performance indicators produced by the middleware. The Grafana throughput panel reports wr/s (writes per second), corresponding to the transaction-throughput measure reported in the text as TPS.

blockchain platform. Nevertheless, an investigation of the blockchain ledger revealed that each transaction reported approximately ± 30 metrics, resulting in a slight but acceptable deviation from the anticipated total number of transactions needed for reporting all 30,000 metrics. While the experiment anticipated precisely 1000 transactions, it actually required 1,030 transactions. We observed that a few transactions did not utilize their maximum capacity to contain metrics in their payloads. Thus, SimBlockLink required an additional 30 transactions to satisfy the total number of metrics (30,000). To help explain the deviation from the anticipated number of transactions, a future study will consider further investigation to determine what led to this phenomenon by examining internal blockchain components and their interconnection, underlying infrastructure and resource allocation, connectivity between the middleware and the blockchain environment, or possibly other causes beyond immediate control.

$$\left(\sum_{i=0}^{n-1} B_i + \sum_{i=0}^{n-1} C_i \right) \stackrel{?}{=} |M|, \quad (6)$$

wherein B_i and C_i are the numbers of breach and compliance records, respectively, associated with threshold requirement v_i ; M is the set of simulator-generated fire-status records; and n is the number of threshold requirements included in the summation. Regardless of the number of total transactions, we verify all generated metrics are submitted to the blockchain platform. For that, we applied Eq. (6) to the state storage, which checks whether all reported B and C sets are identical to the simulator-generated metrics $|M|$, which should be 30,000 in total. This calculation confirmed that all metrics were indeed reported. The observed marginal deviation in the total number of transactions likely represents a minor, which we plan to investigate in a future study.

We hypothesize two potential causes for the observed discrepancy in total transactions. First, as discussed in Sukhwani et al. [68], Hyperledger Fabric's inherent transaction flow might influence the observed behavior. The worker component might be forced to wait until the resolution of its current transaction before initiating the next round. Secondly, the concurrent storage locking mechanism may cause the worker to wait for the lock to be released by the agent, which leads to unexpected delays.

While this experiment validates SimBlockLink's robustness under high load, a head-to-head comparison is not yet possible because no publicly available middleware currently bridges an IoT simulator to a live blockchain (see comparative Table 4, Section 6). As an alternative framework emerge, we will include a direct benchmark in future work.

5.3. Security and privacy considerations

Threat model. SimBlockLink assumes a permissioned Hyperledger Fabric network but must cope with: (i) a *malicious or compromised agent* injecting forged metrics, (ii) a *man-in-the-middle adversary on the TLS-protected gRPC connection that the Fabric JavaSDK opens between the middleware (manager/workers) and Fabric peers/orderers*, (iii) a *denial-of-service attacker targeting the concurrent-storage layer*, and (iv) *leakage of privacy-sensitive IoT payloads written to the ledger*.

Built-in mitigations already present in the implementation.

- *Authenticated, encrypted channels:* workers and agents obtain X.509 certificates via the Fabric Certificate Authority; the SDK then uses those certificates to establish the TLS-secured gRPC channel (see section on identity management), defeating MITM tampering.
- *Lock-based concurrency control:* the segmented locking scheme (Algorithm 3 and Fig. 7) makes each update to the B/C sets atomic, eliminating race-condition exploits and allowing deterministic replay.
- *Fail-safe submission logic:* the retry mechanism in the section "Transactions fail-safe mechanism", provides at-least-once persistence and logs permanent failures, so any data-suppression attempt becomes auditable.
- *Fabric endorsement and access policies:* ledger updates are accepted only from authenticated peers; a rogue agent cannot alter state without satisfying the channel's endorsement policy.

Residual risks and roadmap. The reference implementation currently stores full IoT payloads on-chain; deployments with strict privacy requirements may instead store *only hash pointers* while encrypting the raw data off-chain. Future extensions will therefore: (i) add an off-chain encryption module with on-chain integrity anchors, (ii) formally verify the smart contracts using the Hyperledger Fabric model-checker, and (iii) expose security-relevant metrics (e.g., anomalous lock-starvation counts) via the existing Micrometer/Prometheus pipeline for real-time intrusion dashboards.

6. Related work

The integration of the IoT system with blockchain is a promising advancement that enhances privacy [69,70], security [71], and transparency in data exchange between connected devices, thereby improving the reliability of applications across various domains, such as health-care [69], smart cities [72], and supply chain management [73]. This integration faces technical challenges related to both IoT and blockchain. According to Reyna et al., [74], these challenges include performance, scalability, energy consumption, and security, which pose barriers to the widespread adoption of this technology. Therefore, achieving effective integration between IoT and blockchain requires testing of performance and system design before deployment in real-world environments [75].

Several research efforts have been dedicated to developing evaluation approaches (e.g., simulation, benchmarking) that enable researchers to analyze the performance of integrated systems in controlled environments. This approach enables IoT/Blockchain system simulations under multiple scenarios, and allows for design testing by different configurations to identify shortcomings and address issues before deployment [76]. System complexity and operational requirements may also hinder the evaluation process [29]. For instance, deploying IoT systems for evaluation tasks requires access to large-scale infrastructure and entails significant complexity due to their heterogeneous nature, resulting in high costs and accessibility challenges. IoT simulators have been developed to study system behavior under various operating conditions. For instance, the SimIoT simulator [9] models IoT devices and their interactions with cloud environments to evaluate resource management techniques in IoT-cloud systems. Similarly, the Edge-Fog Cloud architecture [10] focuses on distributed IoT data processing using edge devices and fog nodes, aiming to reduce latency and network congestion.

The iFogSim simulator [45] extends CloudSim [44] to model IoT and edge computing environments, while EdgeNetworkCloudSim [47] offers advanced modeling for edge computing systems, including service-chain placement. MyiFogSim [48], a derivative of iFogSim [45], specializes in virtual machine migration in fog computing environments, emphasizing the user mobility and service quality maintenance.

For distributed fog computing, FogNetSim++ [49] provides unique features such as heterogeneous device modeling, mobility models, fog node scheduling algorithms, and handover mechanisms. Finally, IoTSim-Osmosis [12] extends existing simulators to model IoT applications across the edge-cloud continuum, enabling testing of osmotic computing principles in heterogeneous environments.

Several blockchain simulators have been developed to evaluate decentralized networks. BlockSim [57] is a discrete-event simulator designed to assess blockchain performance (e.g., Bitcoin, Ethereum) under varying configurations. BlockPerf [59], an extension of BlockSim [77], enhances realism by integrating real-world network infrastructure (e.g., network layers) with simulated blockchain layers. Lastly, VIBES [58] simulates the behavior of Bitcoin, Ethereum, and Hyperledger by providing specialized tools for investigating various simulation scenarios.

Since existing simulators operate in isolated environments, focusing either on IoT or blockchain. However, evaluating the feasibility of blockchain-based IoT solutions require access to both domains. Albshri et al. [60] proposed an integrated simulation framework that enables researchers to simulate IoT-blockchain interactions and assess various design choices.

Simulators are often built on simplifications and mathematical assumptions that do not necessarily reflect real-world performance, as they fail to consider unpredictable factors such as network congestion and the impact of system components on performance [39]. For this reason, a set of well-established generic benchmarking tools exists, such as Hyperledger Caliper [63], which can be utilized for simulating an IoT workload. Moreover, some research works highlight the need for IoT-specific benchmarking tools. For instance, BCmaster [62] attempts to realize a modular blockchain performance analyzer for IoT. Neverthe-

Table 4
Comparison of SimBlockLink with related work.

Feature/Study	Simulation (IoT, blockchain, and integration)										Benchmarking tools				Middleware				
	SimIoT [9]	SimIC [56]	iFogSim [45]	EdgeNetwork-CloudSim [47]	MyiFogSim [48]	FogNetSim++ [49]	IoTsim-Osmosis [12]	BlockSim [57]	VIBES [58]	BlockPerf [59]	Alshiri et al. [60]	Blockbench [61]	BCmaster [62]	Caliper [63]	Tapas et al. [64]	Samaniego and Deters [65]	Danish [66]	Zhou et al. [67]	SimBlockLink (Proposed)
Domain	C-IoT	CC	FEC-IoT	EN	MFC	DFC	E2E-IoT	BCT	BCT	BCT	BC-IoT	BCT	BC-IoT	BCT	BC-IoT	BC-IoT	BC-IoT	BC-IoT	BC-IoT
Research Focus	Perf. Eval.	Resource Mgmt. & Perf.	Perf. Eval.	Service-Chain Place.	VM Migr.	Comm. Protoc. & Net. Dyn.	IoT Env.	Consen. & Net.	Peer Net. Behav.	Realistic Perf. Test.	Perf. Eval.	Scalab.	Real-time BC-IoT Eval.	Perf. Eval.	Securing Comm.	Zero-trust	Storage Opt.	Data Sync.	Perf. Eval.
Methodology	Sim	Sim	Sim	Sim	Sim	Sim	Sim	Sim	Sim	Sim	Sim	BM	BM	BM	MW	MW	MW	MW	MW+Sim
IoT Support	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	Partial	✗	✓	✗	✓	✓	✓	✓	✓
Blockchain Support	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Smart Contract	✗	✗	✗	✗	✗	✗	✗	Sim	Sim	Sim	Sim	Sim	✓	✓	✓	✓	✓	✓	✓
Interoperability	Partial	Partial	✗	✗	✗	✗	Partial	✗	✗	✗	Partial	✗	✓	✓	✓	✓	✓	✓	✓
Real-time Data Processing	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓
Concurrency & Race Conditions	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗	✓
Reproducibility	Easy	Easy	Easy	Easy	Easy	Easy	Easy	Easy	Easy	Easy	Easy	Medium	Medium	Medium	Difficult	Difficult	Difficult	Difficult	Easy
Performance Monitoring	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓
Extensibility	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	Partial	Partial	Partial	✗	✗	✗	✗	✓
Open-Source Availability	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓

Domain: *C-IoT* (Cloud and Internet of Things), *CC* (Cloud Computing), *FEC-IoT* (Fog/Edge Computing and Internet of Things), *EN* (Edge Network), *MFC* (Mobile Fog Computing), *DFC* (Distributed Fog Computing), *E2E-IoT* (End-to-End Internet of Things), *BCT* (Blockchain Technology), *BC-IoT* (Blockchain and Internet of Things).

Research Focus: *Perf. Eval.* (Performance Evaluation), *Peer Net. Behav.* (Peer Network Behavior), *Service-Chain Place.* (Service-Chain Placement), *VM Migr.* (Virtual Machine Migration), *Comm. Protoc. & Net. Dyn.* (Communication Protocols & Network Dynamics), *Storage Opt.* (Storage Optimization), *Data Sync.* (Data Synchronization), *IoT Env.* (Internet of Things Environments), *Consen. & Net.* (Consensus & Network), *Resource Mgmt.* (Resource Management), *Realistic Perf. Test.* (Realistic Performance Testing), *Scalab.* (Scalability), *Real-Time BC-IoT Eval.* (Real-Time Blockchain-IoT Evaluation), *Securing Comm.* (Securing Communication), *Zero-Trust* (Zero-Trust Security Model).

Methodology: *Sim* (Simulation), *BM* (Benchmarking), *MW* (Middleware).

less, benchmarking is data-centric and relies on workload assumptions regardless of the IoT's dynamic nature and heterogeneity of IoT applications. The workload settings usually assume prior knowledge of the anticipated IoT application behavior.

Middleware solutions could serve as an alternative approach to overcoming simulation and benchmarking challenges by bridging the gap between simulators and real-world systems. This integration enables researchers to test virtual models within real environments under different operational conditions, validating scalability through real performance evaluations—especially in systems that require configurations and resources that are difficult to acquire [29].

Several research efforts have explored middleware solutions to bridge the gap between IoT and blockchain systems. For instance, Tapas et al. [64] propose a middleware that mainly focuses on securing the communication between IoT applications and the Ethereum-like blockchain platform. Samaniego and Deters [65] also propose a middleware that utilizes a mining-based blockchain to achieve a zero-trust IoT security management mechanism. Danish [66] proposes a middleware that considers IoT requirements for selecting the optimal storage strategy for whether data should be stored on-chain or within cloud-based storage. Wang et al. [78] introduces a middleware that synchronizes data between real IoT applications and Hyperledger Fabric while ensuring coherence and consistency. However, it only focuses on SQL queries and does not provide a rigorous correctness examination, in which failure recovery and concurrent execution should have been considered. Zhou et al. [67] also propose a similar middleware approach. Still, it only concentrates on synchronizing read queries from Hyperledger Fabric to a conventional database while omitting write operations to the ledger.

Despite these efforts, existing middleware solutions primarily focus on connecting real IoT infrastructure to blockchain platforms while neglecting simulation-driven evaluations. This limits their ability to analyze large-scale, experimental blockchain-IoT scenarios under controlled conditions. Moreover, current approaches lack mechanisms for handling interoperability challenges, ensuring synchronization between simulated IoT environments and the real-world blockchain platforms, and providing systematic performance benchmarking. To address these gaps, this paper introduces SimBlockLink, a novel middleware framework that bridges IoT simulators with real blockchain platforms. Unlike prior solutions, SimBlockLink enables realistic performance evaluation by synchronizing simulated IoT workloads with blockchain transactions while mitigating concurrency and race condition issues. This framework is based on a set of technical mechanisms that address challenges such as interoperability, synchronization across different execution environments, and the ability to assess and monitor performance metrics for various design choices in real-world environments under different conditions. Table 4 provides a comparison between SimBlockLink and related works, highlighting the key distinctions and unique contributions of our approach.

7. Conclusions

This paper introduces SimBlockLink, a novel middleware architecture that connects IoT simulators to real blockchain platforms, enabling the evaluation and benchmarking of blockchain performance in simulated IoT scenarios. SimBlockLink is the first middleware framework that enables direct integration between simulated IoT environments and real blockchain platforms, ensuring systematic performance benchmarking under diverse scenarios. Its architecture uniquely incorporates agents to dynamically process simulated IoT workloads, workers to manage blockchain transactions efficiently, and a concurrent storage mechanism to prevent data inconsistencies due to race conditions. The ability to evaluate real blockchain platforms using scalable, configurable IoT workload generators makes SimBlockLink a significant advancement for researchers and practitioners exploring blockchain-based IoT applications.

The case study (a large-scale IoT-based firefighting scenario) demonstrates the middleware's successful integration with Hyperledger Fabric and IoTsim-Osmosis, validating its capability to ensure smooth interoperability and facilitate the evaluation of key blockchain performance metrics. SimBlockLink effectively supports the generation and handling of IoT workloads, demonstrating its potential as a valuable tool for researchers and practitioners.

The current implementation of the middleware focuses on the integration of IoTsim-Osmosis with Hyperledger Fabric. Future work will explore the compatibility of the middleware with other IoT simulators (e.g., Cloudsim [44], iFogSim [45] etc.) and other blockchain platforms (e.g. Quorum [18]).

SimBlockLink offers a robust and scalable framework for evaluating blockchain platforms within IoT simulations. While this study demonstrates its effectiveness with Hyperledger Fabric and a Java-based simulator, extending support to non-Java simulators and diverse blockchain platforms (e.g., Ethereum, Quorum) will further enhance its versatility. Optimizing performance for large-scale simulations will strengthen its applicability across various domains, solidifying SimBlockLink as a foundational tool for blockchain-IoT research. Beyond performance benchmarking, we will integrate the privacy-preserving and formal-verification features outlined in Section 5.3, enabling a holistic security evaluation pipeline.

CRedit authorship contribution statement

Ali Alzubaidi: Software, Methodology, Writing – original draft. **Adel Albshri:** Methodology, Writing – review & editing, Software, Validation, Data curation. **Karan Mitra:** Conceptualization, Writing – review & editing, Supervision. **Rajiv Ranjan:** Supervision. **Ellis Solaiman:** Supervision, Writing – review & editing, Resources, Project administration, Conceptualization.

Funding

This work is funded in part by the EPSRC, under grant number EP/V042017/1 (Scalable Circular Supply Chains for the Built Environment), and in part by the EPSRC, under grant number EP/Y028813/1 (National Edge AI Hub).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] P. Asghari, A.M. Rahmani, H.H.S. Javadi, Internet of things applications: a systematic review, *Comput. Netw.* 148 (2019) 241–261, <https://doi.org/10.1016/j.comnet.2018.12.008>.
- [2] N.K. Tran, M.A. Babar, J. Boan, Integrating blockchain and Internet of things systems: a systematic review on objectives and designs, *J. Netw. Comput. Appl.* 173 (2021) 102844, <https://doi.org/10.1016/j.jnca.2020.102844>.
- [3] A. Alzubaidi, E. Solaiman, P. Patel, K. Mitra, Blockchain-based SLA management in the context of iot, *IT Prof.* 21 (2019) 33–40, <https://doi.org/10.1109/MITP.2019.2909216>.
- [4] K. Wüst, A. Gervais, Do you need a blockchain?, in: *Crypto Valley Conference on Blockchain Technology, CVCBT 2018, IEEE, 2018*, pp. 45–54.
- [5] N. Adhikari, M. Ramkumar, Iot and blockchain integration: applications, opportunities, and challenges, *Netw. Comput. Neural Syst.* 3 (2023) 115–141, <https://doi.org/10.3390/network3010006>.
- [6] D. Pavithran, K. Shaalan, J.N. Al-Karaki, A. Gawanmeh, Towards building a blockchain framework for iot, *Clust. Comput.* 23 (2020) 2089–2103, <https://doi.org/10.1007/s10586-020-03059-5>.
- [7] M. Kaur, S. Gupta, Optimization of a consensus protocol in blockchain-iot convergence, in: *2023 International Conference on Emerging Smart Computing and Informatics (ESCI), IEEE, 2023*, pp. 1–5.
- [8] X. Wang, X. Zha, W. Ni, R.P. Liu, Y.J. Guo, X. Niu, K. Zheng, Survey on blockchain for Internet of things, *Comput. Commun.* 136 (2019) 10–29, <https://doi.org/10.1016/j.comcom.2019.01.006>.

- [9] S. Sotiriadis, N. Bessis, E. Asimakopoulou, N. Mustafee, Towards simulating the Internet of things, in: L. Barolli, K.F. Li, T. Enokido, F. Xhafa, M. Takizawa (Eds.), 28th International Conference on Advanced Information Networking and Applications Workshops, AINA 2014 Workshops, IEEE Computer Society, 2014, pp. 444–448.
- [10] N. Mohan, J. Kangasharju, Edge-fog cloud: a distributed cloud for Internet of things computations, in: 2016 Cloudification of the Internet of Things, CIoT 2016, IEEE, 2016, pp. 1–6.
- [11] C. Sonmez, A. Ozgovde, C. Ersoy, Edgecloudsim: an environment for performance evaluation of edge computing systems, Trans. Emerg. Telecommun. Technol. 29 (2018) e3493, <https://doi.org/10.1002/ett.3493>.
- [12] K. Alwasel, D.N. Jha, F. Habeeb, U. Demirbaga, O.F. Rana, T. Baker, S. Dustdar, M. Villari, P. James, E. Solaiman, R. Ranjan, Iotsim-osmosis: a framework for modeling and simulating iot applications over an edge-cloud continuum, J. Syst. Archit. 116 (2021) 101956, <https://doi.org/10.1016/j.sysarc.2020.101956>.
- [13] M. Chernyshev, Z.A. Baig, O. Bello, S. Zeadally, Internet of things (iot): research, simulators, and testbeds, IEEE Internet Things J. 5 (2018) 1637–1647, <https://doi.org/10.1109/JIOT.2017.2786639>.
- [14] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A.D. Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolic, S.W. Cocco, J. Yellick, Hyperledger fabric: a distributed operating system for permissioned blockchains, in: R. Oliveira, P. Felber, Y.C. Hu (Eds.), Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, ACM, 2018, 30.
- [15] A. Kumar, B.K. Sah, T. Mehrotra, G.K. Rajput, A review on double spending problem in blockchain, in: 2023 International Conference on Computational Intelligence and Sustainable Engineering Solutions (CISES), IEEE, 2023, pp. 881–889.
- [16] S. Nakamoto, Bitcoin: a peer-to-peer electronic cash system, www.bitcoin.org, 2008.
- [17] V. Buterin, A next-generation smart contract and decentralized application platform, http://www.the-blockchain.com/docs/Ethereum_white_paper-a_next_generation_smart_contract_and_decentralized_application_platform-vitalik-buterin.pdf, 2014. (Accessed 21 August 2026).
- [18] J.P. Morgan, A permissioned implementation of Ethereum supporting data privacy, <https://github.com/Consensys/quorum>, 2016. (Accessed 21 August 2026).
- [19] M.K. Shrivastava, T. Yeboah, The disruptive blockchain: types, platforms and applications, Texila Int. J. Acad. Res. 3 (2019) 17–39, <https://doi.org/10.21522/TIJAR.2014.SE.19.01.Art003>.
- [20] M. Piteck, V. Podobnik, G. Jezic, Beyond the Internet of things: the social networking of machines, Int. J. Distrib. Sens. Netw. 12 (2016) 8178417, <https://doi.org/10.1155/2016/8178417>.
- [21] W. Yan, Z. Wang, H. Wang, W. Wang, J. Li, X. Gui, Survey on recent smart gateways for smart home: systems, technologies, and challenges, Trans. Emerg. Telecommun. Technol. 33 (2022) e4067, <https://doi.org/10.1002/ett.4067>.
- [22] J.J. Peralta Abadía, C. Walther, A. Osman, K. Smarsly, A systematic survey of Internet of things frameworks for smart city applications, Sustain. Cities Soc. 83 (2022) 103949, <https://doi.org/10.1016/j.scs.2022.103949>.
- [23] Z. Alavikia, M. Shabro, A comprehensive layered approach for implementing Internet of things-enabled smart grid: a survey, Digit. Commun. Netw. 8 (2022) 388–410, <https://doi.org/10.1016/j.dcan.2022.01.002>.
- [24] A.I. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of things: a survey on enabling technologies, protocols, and applications, IEEE Commun. Surv. Tutor. 17 (2015) 2347–2376, <https://doi.org/10.1109/COMST.2015.2444095>.
- [25] M.R. Bouakouk, A. Abdelli, L. Mokdad, Survey on the cloud-iot paradigms: taxonomy and architectures, in: IEEE Symposium on Computers and Communications, ISCC 2020, IEEE, 2020, pp. 1–6.
- [26] R. Ait Mouha, Internet of things (IoT), J. Data Anal. Inf. Process. 09 (2021) 77–101, <https://doi.org/10.4236/jdaip.2021.92006>.
- [27] A.A. Sadawi, M.S. Hassan, M.M. Ndiaye, A survey on the integration of blockchain with iot to enhance performance and eliminate challenges, IEEE Access 9 (2021) 54478–54497, <https://doi.org/10.1109/ACCESS.2021.3070555>.
- [28] D. Anandayuvraj, J.C. Davis, Reflecting on recurring failures in iot development, in: 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10–14, 2022, ACM, 2022, 185.
- [29] M.A. Uddin, A. Stranieri, I. Gondal, V. Balasubramanian, A survey on the adoption of blockchain in iot: challenges and solutions, Blockchain Res. Appl. 2 (2021) 100006, <https://doi.org/10.1016/j.bcr.2021.100006>.
- [30] S. Kar, B. Chakravorty, S. Sinha, M. Gupta, Analysis of stakeholders within iot ecosystem, in: Digital India: Reflections and Practice, Springer, Cham, 2018, pp. 251–276.
- [31] H.R. Hasan, E. Alhadhrami, A. Aldaheri, K. Salah, R. Jayaraman, Smart contract-based approach for efficient shipment management, Comput. Ind. Eng. 136 (2019) 149–159, <https://doi.org/10.1016/j.cie.2019.07.022>.
- [32] C. Kolias, G. Kambourakis, A. Stavrou, J.M. Voas, Ddos in the iot: Mirai and other botnets, Computer 50 (2017) 80–84, <https://doi.org/10.1109/MC.2017.201>.
- [33] S. Kikuchi, T. Aoki, Evaluation of operational vulnerability in cloud service management using model checking, in: Seventh IEEE International Symposium on Service-Oriented System Engineering, SOSE 2013, IEEE Computer Society, 2013, pp. 37–48.
- [34] A. Kuznetsov, I. Oleshko, V. Tymchenko, K. Lisitsky, M. Rodinko, A. Kolhatin, Performance analysis of cryptographic hash functions suitable for use in blockchain, Int. J. Comput. Netw. Inf. Secur. 13 (2021) 1–15, <https://doi.org/10.5815/ijcnis.2021.02.01>.
- [35] L. Alevizos, V. Ta, M.H. Eiza, Augmenting zero trust architecture to endpoints using blockchain: a state-of-the-art review, Secur. Priv. 5 (2022) e191, <https://doi.org/10.1002/spy2.191>.
- [36] I. Makhdoom, M. Abolhasan, H. Abbas, W. Ni, Blockchain's adoption in iot: the challenges, and a way forward, J. Netw. Comput. Appl. 125 (2019) 251–279, <https://doi.org/10.1016/j.jnca.2018.10.019>.
- [37] A. Albshri, B. Awaji, E. Solaiman, Investigating the requirement of building blockchain simulator for iot applications, in: IEEE International Conference on Smart Internet of Things, SmartIoT 2022, Suzhou, China, August 19–21, 2022, IEEE, 2022, pp. 232–240.
- [38] M. Alaslani, F. Nawab, B. Shihada, Blockchain in iot systems: end-to-end delay evaluation, IEEE Internet Things J. 6 (2019) 8332–8344, <https://doi.org/10.1109/JIOT.2019.2917226>.
- [39] A. Maria, Introduction to modeling and simulation, in: S. Andradóttir, K.J. Healy, D.H. Withers, B.L. Nelson (Eds.), Proceedings of the 29th Conference on Winter Simulation, WSC 1997, ACM, 1997, pp. 7–13.
- [40] D.R. Anderson, D.J. Sweeney, T.A. Williams, J.D. Camm, J.J. Cochran, An introduction to management science: quantitative approach, Cengage Learning, Boston, 2018.
- [41] R.R. Hill, Discrete-event simulation: a first course, J. Simul. 1 (2007) 147–148, <https://doi.org/10.1057/palgrave.jos.4250012>.
- [42] B.R. Haverkort, Performance of Computer Communication Systems - a Model-Based Approach, Wiley, New York, 1998.
- [43] A. Márkus, A. Kertész, A survey and taxonomy of simulation environments modelling fog computing, Simul. Model. Pract. Theory 101 (2020) 102042, <https://doi.org/10.1016/j.simp.2019.102042>.
- [44] R.N. Calheiros, R. Ranjan, A. Beloglazov, C.A.F.D. Rose, R. Buyya, Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms, Softw. Pract. Exp. 41 (2011) 23–50, <https://doi.org/10.1002/spe.995>.
- [45] H. Gupta, A.V. Dastjerdi, S.K. Ghosh, R. Buyya, ifogsim: a toolkit for modeling and simulation of resource management techniques in the Internet of things, edge and fog computing environments, Softw. Pract. Exp. 47 (2017) 1275–1296, <https://doi.org/10.1002/spe.2509>.
- [46] D.N. Jha, K. Alwasel, A. Alshoshan, X. Huang, R.K. Naha, S.K. Battula, S. Garg, D. Puthal, P. James, A.Y. Zomaya, S. Dustdar, R. Ranjan, Iotsim-edge: a simulation framework for modeling the behavior of Internet of things and edge computing environments, Softw. Pract. Exp. 50 (2020) 844–867, <https://doi.org/10.1002/spe.2787>.
- [47] M. Seufert, B.K. Kwam, F. Wamser, P. Tran-Gia, Edgenetworkcloudsim: placement of service chains in edge clouds using networkcloudsim, in: 2017 IEEE Conference on Network Softwarization, NetSoft 2017, IEEE, 2017, pp. 1–6.
- [48] M.M. Lopes, W.A. Higashino, M.A. Capretz, L.F. Bittencourt, Myifogsim: a simulator for virtual machine migration in fog computing, in: Companion Proceedings of The10th International Conference on Utility and Cloud Computing, Association for Computing Machinery, New York, 2017, pp. 47–52.
- [49] T. Qayyum, A.W. Malik, M.A.K. Khattak, O. Khalid, S.U. Khan, Fognetsim++: a toolkit for modeling and simulation of distributed fog environment, IEEE Access 6 (2018) 63570–63583, <https://doi.org/10.1109/ACCESS.2018.2877696>.
- [50] R.E. Schantz, D.C. Schmidt, Middleware for distributed systems, in: B.W. Wah (Ed.), Wiley Encyclopedia of Computer Science and Engineering, John Wiley & Sons, Inc., 2008.
- [51] IBM, Middleware, <https://www.ibm.com/topics/middleware>, 2024. (Accessed 21 August 2026).
- [52] Hyperledger, Hyperledger blockchain performance metrics, Technical Report. Hyperledger Performance and Scale Working Group, <https://www.hyperledger.org/learn/publications/blockchain-performance-metrics>, 2018. (Accessed 21 August 2026).
- [53] Prometheus Authors, Prometheus - monitoring system & time series database, <https://prometheus.io>, 2024. (Accessed 12 June 2024).
- [54] Grafana Labs, Grafana: the open observability platform, <https://grafana.com>, 2024. (Accessed 12 June 2024).
- [55] D. Ongaro, J. Ousterhout, In search of an understandable consensus algorithm, in: 2014 USENIX Annual Technical Conference (USENIX ATC 14), USENIX Association, 2014, pp. 305–319.
- [56] S. Sotiriadis, N. Bessis, N. Antonopoulos, A. Anjum, Simic: designing a new inter-cloud simulation platform for integrating large-scale resource management, in: 2013 IEEE 27th International Conference on Advanced Information Networking and Applications (AINA), IEEE, 2013, pp. 90–97.
- [57] M. Alharby, A. van Moorsel, Blocksims: an extensible simulation tool for blockchain systems, Front. Blockchain 3 (2020) 28, <https://doi.org/10.3389/fbloc.2020.00028>.
- [58] L. Stoykov, K. Zhang, H.A. Jacobsen, Vibes: fast blockchain simulations for large-scale peer-to-peer networks: demo, in: Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference: Posters and Demos, Association for Computing Machinery, 2017, pp. 19–20.
- [59] J. Polge, S. Ghatpande, S. Kubler, J. Robert, Y.L. Traon, Blockperf: a hybrid blockchain emulator/simulator framework, IEEE Access 9 (2021) 107858–107872, <https://doi.org/10.1109/ACCESS.2021.3101044>.
- [60] A. Albshri, A. Alzubaidi, M. Alharby, B. Awaji, K. Mitra, E. Solaiman, A conceptual architecture for simulating blockchain-based iot ecosystems, J. Cloud Comput. 12 (2023) 103, <https://doi.org/10.1186/s13677-023-00481-z>.

- [61] T.T.A. Dinh, J. Wang, G. Chen, R. Liu, B.C. Ooi, K. Tan, BLOCKBENCH: a framework for analyzing private blockchains, in: S. Salihoglu, W. Zhou, R. Chirkova, J. Yang, D. Suciu (Eds.), *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017*, ACM, 2017, pp. 1085–1100.
- [62] Y. Liu, K. Qian, K. Wang, L. He, BCmaster: a compatible framework for comprehensively analyzing and monitoring blockchain systems in IoT, *IEEE Internet Things J.* 9 (2022) 22529–22546, <https://doi.org/10.1109/JIOT.2022.3182004>.
- [63] Hyperledger, Hyperledger caliper, <https://hyperledger.github.io/caliper/>. (Accessed 21 August 2026).
- [64] N. Tapas, G. Merlino, F. Longo, Blockchain-based iot-cloud authorization and delegation, in: *2018 IEEE International Conference on Smart Computing, SMARTCOMP 2018*, Taormina, Sicily, Italy, June 18–20, 2018, IEEE Computer Society, 2018, pp. 411–416.
- [65] M. Samaniego, R. Deters, Zero-trust hierarchical management in iot, in: *2018 IEEE International Congress on Internet of Things, ICIOT 2018*, San Francisco, CA, USA, July 2–7, 2018, IEEE Computer Society, 2018, pp. 88–95.
- [66] S.M. Danish, A blockchain-based adaptive middleware for large scale Internet of things data storage selection, in: F. Nawab, E. Rivière (Eds.), *Proceedings of the 20th International Middleware Conference Doctoral Symposium, Middleware 2019*, ACM, 2019, pp. 17–19.
- [67] E. Zhou, H. Sun, B. Pi, J. Sun, K. Yamashita, Y. Nomura, Ledgerdata refiner: a powerful ledger data query platform for hyperledger fabric, in: M.A. Alsmirat, Y. Jararweh (Eds.), *Sixth International Conference on Internet of Things: Systems, Management and Security, IOTSMS 2019*, IEEE, 2019, pp. 433–440.
- [68] H. Sukhwani, N. Wang, K.S. Trivedi, A.J. Rindos, Performance modeling of hyperledger fabric (permissioned blockchain network), in: *17th IEEE International Symposium on Network Computing and Applications, NCA 2018*, IEEE, 2018, pp. 1–8.
- [69] C. Dhasaratha, M.K. Hasan, S. Islam, S. Khapre, S. Abdullah, T.M. Ghazal, A.I. Alzahrani, N. Alalwan, N. Vo, M. Akhtaruzzaman, Data privacy model using blockchain reinforcement federated learning approach for scalable Internet of medical things, *CAAI Trans. Intell. Technol.* (2024), <https://doi.org/10.1049/cit2.12287>.
- [70] A. Dorri, S.S. Kanhere, R. Jurdak, P. Gauravaram, Blockchain for iot security and privacy: the case study of a smart home, in: *2017 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, IEEE, 2017, pp. 618–623.
- [71] K. Christidis, M. Devetsikiotis, Blockchains and smart contracts for the Internet of things, *IEEE Access* 4 (2016) 2292–2303, <https://doi.org/10.1109/ACCESS.2016.2566339>.
- [72] J. Sun, J. Yan, K.Z. Zhang, Blockchain-based sharing services: what blockchain technology can contribute to smart cities, *Financ. Innov.* 2 (2016) 1–9, <https://doi.org/10.1186/s40854-016-0040-y>.
- [73] S. Saberi, M. Kouhizadeh, J. Sarkis, L. Shen, Blockchain technology and its relationships to sustainable supply chain management, *Int. J. Prod. Res.* 57 (2019) 2117–2135, <https://doi.org/10.1080/00207543.2018.1533261>.
- [74] A. Reyna, C. Martín, J. Chen, E. Soler, M. Díaz, On blockchain and its integration with iot. Challenges and opportunities, *Future Gener. Comput. Syst.* 88 (2018) 173–190, <https://doi.org/10.1016/j.future.2018.05.046>.
- [75] M. Conoscenti, A. Vetro, J.C. De Martin, Blockchain for the Internet of things: a systematic literature review, in: *2016 IEEE/ACS 13th International Conference of Computer Systems and Applications (AICCSA)*, IEEE, 2016, pp. 1–6.
- [76] J. Zheng, C. Dike, S. Pancari, Y. Wang, G.C. Giakos, W. Elmannai, B. Wei, An in-depth review on blockchain simulators for iot environments, *Future Internet* 14 (2022) 182, <https://doi.org/10.3390/fi14060182>.
- [77] C. Faria, M. Correia, Blocksimsim: blockchain simulator, in: *IEEE International Conference on Blockchain, Blockchain 2019*, IEEE, 2019, pp. 439–446.
- [78] N. Wang, B. Wang, T. Liu, W. Li, S. Yang, A middleware approach to synchronize transaction data to blockchain, in: *29th International Conference on Computer Communications and Networks, ICCCN 2020*, Honolulu, HI, USA, August 3–6, 2020, IEEE, 2020, pp. 1–8.